Introduction to Machine Learning

Machine Learning (ML) is a branch of Artificial Intelligence (AI) that enables computers to learn patterns from data and make decisions without being explicitly programmed. Instead of following rigid rules, ML models identify patterns and improve performance over time as they process more data.

Why is Machine Learning Important?

ML is widely used across industries for tasks such as:

- Predictive analytics: Stock market trends, weather forecasting.
- Automation: Self-driving cars, robotics.
- Healthcare: Disease diagnosis, drug discovery.
- Finance: Fraud detection, credit risk analysis.
- Customer experience: Chatbots, recommendation systems (Netflix, Amazon).

Types of Machine Learning

1. **Supervised Learning**
    - The model learns from labeled data (input-output pairs).
    - Example: Predicting house prices based on features like size, location, etc.
    - Algorithms: Linear Regression, Decision Trees, Random Forest, Support Vector Machines (SVM), Neural Networks.
2. **Unsupervised Learning**
    - The model finds patterns in unlabeled data.
    - Example: Customer segmentation in marketing.
    - Algorithms: K-Means Clustering, Principal Component Analysis (PCA), DBSCAN.

3. **Reinforcement Learning**
    - The model learns by interacting with an environment and receiving rewards or penalties.
    - Example: AlphaGo (Google's AI that defeated human Go players).
    - Algorithms: Q-Learning, Deep Q Networks (DQN).

Key Components of Machine Learning

- Data: High-quality and diverse datasets improve model accuracy.
- Features: Attributes used for learning patterns.
- Algorithms: Methods to learn from data and make predictions.
- Model Evaluation: Metrics like accuracy, precision, recall, and F1-score to assess performance.

Challenges in Machine Learning

- Overfitting: The model memorizes training data instead of generalizing.
- Bias-Variance Tradeoff: Balancing underfitting and overfitting.
- Data Quality: Noisy, incomplete, or imbalanced data can affect model performance.
- Computational Complexity: Training deep learning models requires powerful hardware.

Machine Learning is transforming industries by enabling automation, decision-making, and data-driven insights. With advances in deep learning and neural networks, ML continues to evolve, making systems smarter and more efficient.

Introduction to Data Visualization

Data visualization is the process of representing data graphically to help understand trends, patterns, and insights effectively. It uses visual elements such as charts, graphs, and maps to make complex data more accessible and easier to interpret.

Why is Data Visualization Important?

- **Improves understanding**: Helps in identifying patterns and trends that may not be noticeable in raw data.
- **Enhances decision-making**: Supports better business and research decisions by presenting data clearly.
- **Simplifies complex information**: Large datasets become easier to interpret.
- **Facilitates communication**: Helps convey insights effectively to stakeholders.

Types of Data Visualization

1. **Univariate Visualization (Single Variable Analysis)**
    - Histogram: Shows frequency distribution.
    - Box Plot: Identifies outliers and spread of data.
2. **Bivariate and Multivariate Visualization (Relationship Between Variables)**
    - Scatter Plot: Shows correlation between two variables.
    - Heatmap: Represents correlation between multiple variables using color gradients.
3. **Categorical Data Visualization**
    - Bar Chart: Compares categorical data.
    - Pie Chart: Shows proportions.
4. **Time Series Visualization**

- Line Chart: Tracks trends over time (e.g., stock prices, temperature changes).

5. **Geospatial Visualization**
    - Maps: Used for geographic data representation (e.g., population density).

Popular Data Visualization Libraries

- **Python**: Matplotlib, Seaborn, Plotly
- **R**: ggplot2, Shiny
- **BI Tools**: Tableau, Power BI

Data visualization plays a crucial role in data analysis and decision-making by making information more intuitive and accessible. Choosing the right visualization techniques ensures accurate and meaningful representation of data.

Introduction to PyCharm

PyCharm is a powerful integrated development environment (IDE) for Python, developed by **JetBrains**. It provides essential tools for efficient Python development, including code analysis, debugging, testing, and version control.

Why Use PyCharm?

- **Intelligent Code Assistance**: Provides code completion, syntax highlighting, and error detection.
- **Built-in Debugger**: Helps in identifying and fixing errors.
- **Version Control Integration**: Supports Git, SVN, and other VCS tools.
- **Web Development Support**: Works with Django, Flask, and other frameworks.
- **Database Tools**: Directly connects to databases for easy data handling.
- **Extensive Plugin Support**: Can be customized using various plugins.

Features of PyCharm

1. **Code Editor**
   - Syntax highlighting, auto-completion, and PEP8 compliance.
2. **Debugger**
   - Built-in graphical debugger for step-by-step execution.
3. **Virtual Environments & Package Management**
   - Supports venv, conda, and package installation using pip.
4. **Integration with Web Frameworks**
   - Works seamlessly with Django, Flask, and FastAPI.
5. **Refactoring & Productivity Tools**
   - Renaming, extracting variables, and other code improvements.

6. **Testing Frameworks**
    - Supports unittest, pytest, and doctest.
7. **Jupyter Notebook Support**
    - Allows running Jupyter notebooks inside PyCharm.

Editions of PyCharm

- **PyCharm Community (Free)**: Basic features for Python development.
- **PyCharm Professional (Paid)**: Advanced tools for web development, databases, and scientific computing.

How to Install PyCharm?

1. Download PyCharm from JetBrains Official Website.
2. Install the software and configure the Python interpreter.
3. Start a new project and begin coding!

PyCharm is a feature-rich IDE that enhances Python development with powerful tools and automation. It is widely used by professionals and beginners for software development, data science, and web applications.

1.Develop a program to create histograms for all numerical features and analyse the distribution of each feature. Generate box plots for all numerical features and identify any outliers. Use California Housing dataset.

AIM:

To perform an exploratory data analysis (EDA) on the California Housing dataset, focusing on the numerical features.

OBJECTIVES:

- **Load the Dataset:** Retrieve the California Housing dataset using the fetch_california_housing function from the sklearn.datasets module and load it into a Pandas DataFrame.
- **Create Histograms:** Generate histograms with KDE (Kernel Density Estimate) for each numerical feature to visualize their distributions.
- **Generate Box Plots:** Create box plots for each numerical feature to visualize the spread and identify potential outliers.
- **Detect Outliers:** Use the Interquartile Range (IQR) method to detect outliers in each numerical feature and summarize the number of outliers detected.
- **Dataset Summary:** Optionally print a statistical summary of the dataset using the describe method to provide an overview of the key statistics for each numerical feature.

PROGRAM:

#import necessary libraries

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
```

```
#Load the California Housing dataset
data = fetch_california_housing(as_frame=True)
housing_df = data.frame

# Selecting Numerical Features
numerical_features = housing_df.select_dtypes(include=[np.number]).columns

# Plot Histograms for Numerical Features
plt.figure(figsize=(15, 10))
for i, feature in enumerate(numerical_features):
    plt.subplot(3, 3, i + 1)
    sns.histplot(housing_df[feature], kde=True, bins=30, color='blue')
    plt.title(f'Distribution of {feature}')
plt.tight_layout()
plt.show()

# Generating Box Plots for Numerical Features
plt.figure(figsize=(15, 10))
for i, feature in enumerate(numerical_features):
    plt.subplot(3, 3, i + 1)
    sns.boxplot(x=housing_df[feature], color='orange')
    plt.title(f'Box Plot of {feature}')
plt.tight_layout()
plt.show()

# Identifying Outliers Using the Interquartile Range (IQR) Method
print("Outliers Detection:")
outliers_summary = {}
for feature in numerical_features:
    Q1 = housing_df[feature].quantile(0.25)
    Q3 = housing_df[feature].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
```

```
    upper_bound = Q3 + 1.5 * IQR
    outliers = housing_df[(housing_df[feature] < lower_bound) |
                          (housing_df[feature] > upper_bound)]
    outliers_summary[feature] = len(outliers)
    print(f"{feature}: {len(outliers)} outliers")
```

#Print a summary of the dataset

```
print("\nDataset Summary:")
print(housing_df.describe())
```

EXPLANATION:

Importing Necessary Libraries

- **import pandas as pd:** Used for handling structured data (dataframes, series).
- **import numpy as np:** Helps with numerical operations and array manipulations.
- **import seaborn as sns:** Enhances data visualization (histograms, box plots).
- **import matplotlib.pyplot as plt:** Used for plotting figures and customizing visualizations.
- **from sklearn.datasets import fetch_california_housing:** This imports the fetch_california_housing function from sklearn.datasets**.** fetch_california_housing is used to load the California housing dataset, which contains real estate data from California, including information like median house values, population, and location-based attributes.

Loading the California Housing Dataset

- **data = fetch_california_housing(as_frame=True):** Loads the dataset as a Pandas DataFrame.
- **housing_df = data.frame:** data.frame extracts the actual data table into housing_df. housing_df is now a Pandas DataFrame containing California housing data.

Selecting Numerical Features

numerical_features = housing_df.select_dtypes(include=[np.number]).columns:

- select_dtypes(include=[np.number]) selects only numerical columns.
- .columns extracts their names.

- numerical_features now holds all the numeric feature names.

Plotting Histograms for Numerical Features

This block creates histograms for each numerical feature:

- **plt.figure(figsize=(15, 10))** initializes a figure with a specified size.
- A **for** loop iterates through each numerical feature.
- **plt.subplot(3, 3, i + 1)** creates a subplot in a 3x3 grid.
- **sns.histplot** generates a histogram with Kernel Density Estimation (KDE) for the given feature.
- **plt.title** sets the title for each subplot.
- **plt.tight_layout** adjusts subplots to fit into the figure area.
- **plt.show** displays the histograms.

Generating Box Plots for Numerical Features

This block generates box plots for each numerical feature:

- **plt.figure(figsize=(15, 10))** initializes a figure with a specified size.
- A **for** loop iterates through each numerical feature.
- **plt.subplot(3, 3, i + 1)** creates a subplot in a 3x3 grid.
- **sns.boxplot** generates a box plot for the given feature.
- **plt.title** sets the title for each subplot.
- **plt.tight_layout** adjusts subplots to fit into the figure area.
- **plt.show** displays the box plots.

Identifying Outliers Using the Interquartile Range (IQR) Method

This block identifies outliers using the Interquartile Range (IQR) method:

- **print ("Outliers Detection:")** prints a header for the outlier detection section.
- **outliers_summary** = {} initializes an empty dictionary **outliers_summary** to store the number of outliers for each feature.
- **for feature in numerical_features**: A **for** loop iterates through each numerical feature.

- **Q1 = housing_df[feature].quantile(0.25)** calculates the first quartile (Q1) for each feature.
- **Q3 = housing_df[feature].quantile(0.75)** calculates the third quartile (Q3) for each feature.
- **IQR = Q3 - Q1** calculates the IQR as **Q3 - Q1.**
- **lower_bound = Q1 - 1.5 * IQR** determines the lower bound for outlier detection.
- **upper_bound = Q3 + 1.5 * IQR** determines the upper bounds for outlier detection.
- **outliers = housing_df[(housing_df[feature] < lower_bound) | (housing_df[feature] > upper_bound)]** identifies outliers as values below the lower bound or above the upper bound.
- **outliers_summary[feature] = len(outliers)** stores the number of outliers in outliers_summary and prints the count.
- **print(f"{feature}: {len(outliers)} outliers")** prints the outlier count

Displaying a Summary of the Dataset

This block prints a summary of the dataset:

- **print("\nDataset Summary:")** prints a header for the dataset summary section.
- **housing_df.describe()** provides a descriptive statistics summary for the numerical features, and print displays it.

IQR (Interquartile Range): The Interquartile Range (IQR) is a measure of statistical dispersion, representing the range within which the middle 50% of values in a dataset lie. It is used to detect outliers and understand the spread of the data.

Formula: **IQR=Q3−Q1**

Where:

Q1 (First Quartile / 25th Percentile) → The value below which 25% of the data falls.

Q3 (Third Quartile / 75th Percentile) → The value below which 75% of the data falls.

IQR → The range between Q1 and Q3 (middle 50% of data).

How is IQR Used to Detect Outliers? An outlier is a value that is significantly lower or higher than the rest of the data. Outliers are typically found using this rule:

- Lower Bound = Q1−1.5×IQRQ1 - 1.5 \times IQRQ1−1.5×IQR
- Upper Bound = Q3+1.5×IQRQ3 + 1.5 \times IQRQ3+1.5×IQR

Any value below the lower bound or above the upper bound is considered an outlier.

Example Calculation:

Let's say we have the following dataset:

[10,15,20,25,30,35,40,100][10, 15, 20, 25, 30, 35, 40, 100][10,15,20,25,30,35,40,100]

Q1 (25th percentile) = 17.5

Q3 (75th percentile) = 37.5

IQR = Q3−Q1=37.5−17.5=20

Q3 - Q1 = 37.5 - 17.5 = 20

Q3−Q1=37.5−17.5=20

Lower Bound = 17.5−(1.5×20)=−12.517.5 - (1.5 \times 20) = -12.517.5−(1.5×20)=−12.5
Upper Bound = 37.5+(1.5×20)=67.537.5 + (1.5 \times 20) = 67.537.5+(1.5×20)=67.5

Outliers:

- The value 100 is greater than 67.5, so it is an outlier.

Why is IQR Useful?

- Helps detect **outliers** that could distort analysis.
- Less affected by extreme values than the **mean and standard deviation**.
- Commonly used in **box plots** to visualize the spread of data.

OUTPUT:

Figure1 contains histograms of various numerical features from the California Housing dataset. Following is an analysis of each distribution:

- **MedInc (Median Income)**

The distribution is right-skewed (positively skewed), meaning most households have a lower income, but a few have very high incomes.There is a long tail towards the right.

- **HouseAge (Median House Age)**

  The distribution appears bimodal, with peaks around 20-30 years and another at 50 years. This suggests that many houses were built in specific periods.

- **AveRooms (Average Rooms per Household)**

  The distribution is heavily right-skewed. Most values are concentrated near 2–5 rooms, but some extreme values suggest outliers (e.g., houses with 100+ rooms).

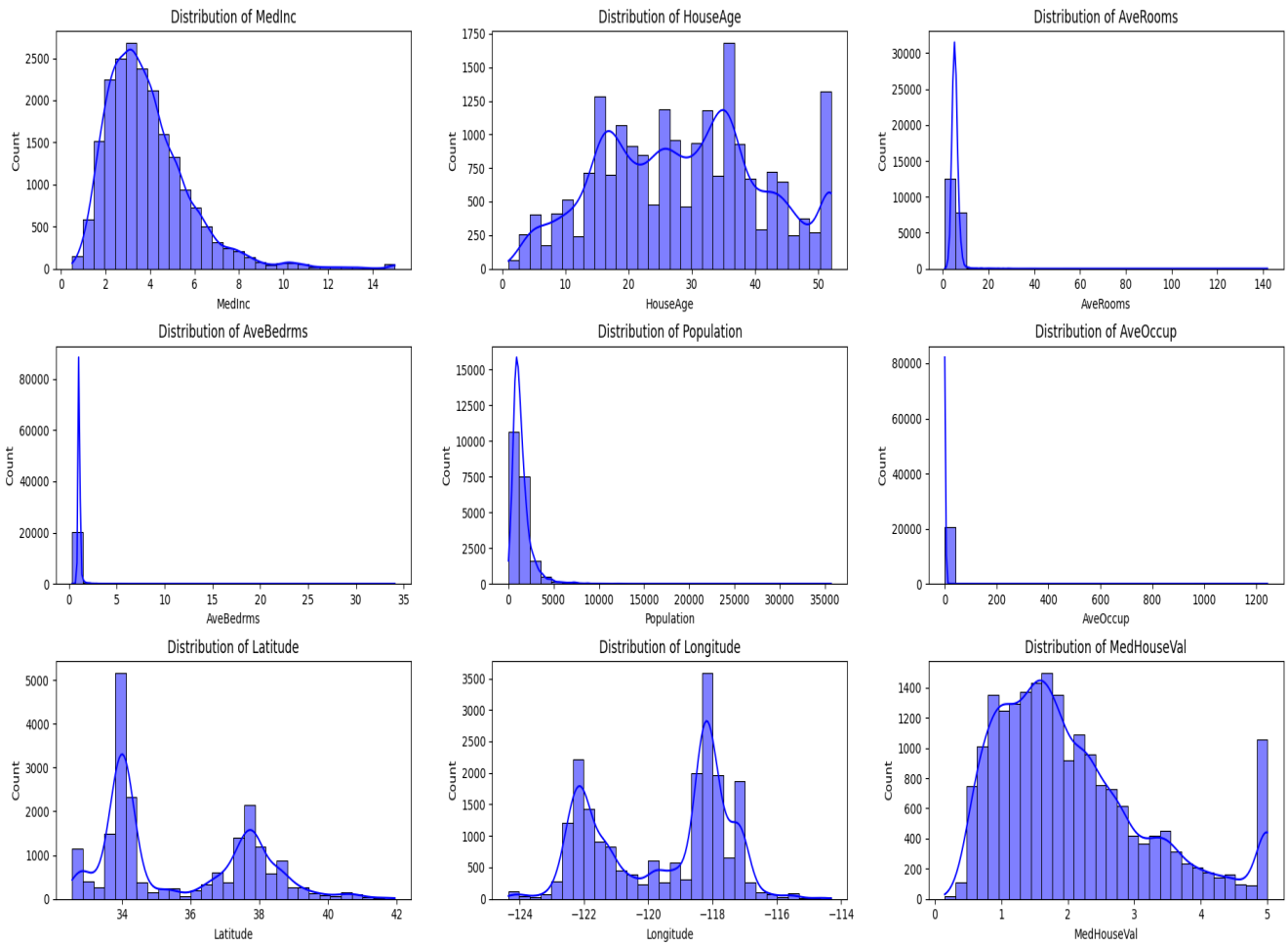


Figure1.1: Histogram For Numeric Values

- **AveBedrms (Average Bedrooms per Household)**

  Like AveRooms, it is right-skewed. Most houses have 1–3 bedrooms, but there are extreme values that may indicate data anomalies.

- **Population (Household Population in a Block)**

  The distribution has a large concentration near the lower end, but there are very high values. Some areas have populations in the thousands, indicating high-density areas.
- **AveOccup (Average Occupants per Household)**

  The distribution is highly skewed. Most households have 1–3 occupants, but some values are extremely high (e.g., over 1000+ occupants), which are likely outliers.
- **Latitude & Longitude**

  These represent geographical locations of houses in California. The distributions show different density clusters, which likely correspond to urban vs. rural areas.
- **MedHouseVal (Median House Value)**

  The distribution is right-skewed, indicating that most house values are on the lower end, but a few houses are very expensive. The peak at the highest value suggests capped data (a limit in the dataset, possibly $500,000).

Key Observations

- **Right-skewed distributions:** Most features, like income, rooms, and population, have long tails, indicating extreme values.
- **Potential outliers:** Features like AveRooms, AveBedrms, AveOccup have extreme values that need investigation.
- **Bimodal trend in House Age:** Suggests distinct construction periods in California's housing market.
- **Capped values**: House values appear to be limited at the higher end.

Figure2 contains **box plots** of numerical features from the **California Housing dataset.** Box plots help visualize **outliers** and **data distribution**:

- **MedInc (Median Income)**

  The box is centered, meaning the data is fairly symmetric. There are many outliers on the right side, indicating a few households with very high income.
- **HouseAge (Median House Age)**

  The data is more evenly distributed. No major outliers, suggesting house age values are within a reasonable range.
- **AveRooms (Average Rooms per Household)**

The box is very narrow, meaning most data is tightly packed. Many extreme outliers, indicating that some households report unrealistically high average rooms (e.g., 100+ rooms).

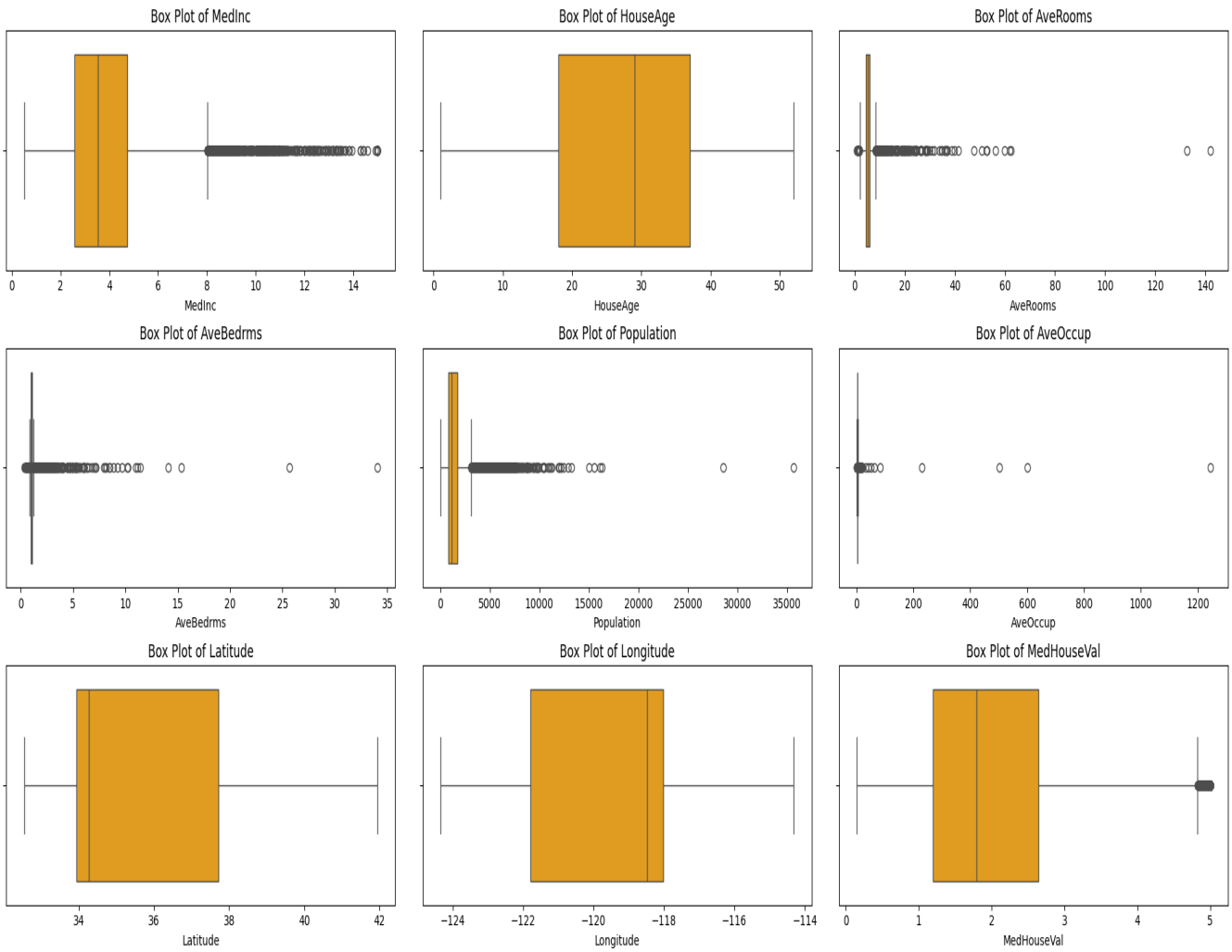


Figure1.2: Box Plots for Numerical Features

- **AveBedrms (Average Bedrooms per Household)**

  Similar to AveRooms, it has a narrow range but many outliers. Some extreme values (e.g., >10 bedrooms) suggest potential data errors or unusual properties.

- **Population (Household Population in a Block)**

  Highly skewed, with a few blocks having massive populations (>25,000). These extreme values might indicate high-density areas or recording issues.

- **AveOccup (Average Occupants per Household)**

Extremely skewed. Most values are small, but some extreme values exceed 1000, which is likely a data anomaly.

- **Latitude & Longitude**

  These geographical features have a wider range and no significant outliers.This is expected since they represent locations in California.

- **MedHouseVal (Median House Value)**

  The upper limit has many outliers, indicating a capped dataset (house values may be restricted, possibly at $500,000). Suggests many high-value properties were grouped at this limit.

Key Observations

- **Severe outliers:** Found in AveRooms, AveBedrms, Population, and AveOccup—indicating possible data inconsistencies or extreme values.
- **Capped values:** MedHouseVal seems to have a hard limit, possibly due to dataset constraints.
- **HouseAge is well-distributed:** No major outliers, indicating a natural spread of house ages.
- **Geographical data (Latitude & Longitude) is normal:** No anomalies.

Data summary:

```
C:\Users\Nalini\PycharmProjects\pythonProject1\.venv\Scripts\python.exe C:\Users\Nalini\PycharmProjects\pythonProject1\ml1pgm.py
Outliers Detection:
MedInc: 681 outliers

Dataset Summary:
             MedInc      HouseAge  ...     Longitude   MedHouseVal
count  20640.000000  20640.000000  ...  20640.000000  20640.000000
mean       3.870671     28.639486  ...   -119.569704      2.068558
std        1.899822     12.585558  ...      2.003532      1.153956
min        0.499900      1.000000  ...   -124.350000      0.149990
25%        2.563400     18.000000  ...   -121.800000      1.196000
50%        3.534800     29.000000  ...   -118.490000      1.797000
75%        4.743250     37.000000  ...   -118.010000      2.647250
max       15.000100     52.000000  ...   -114.310000      5.000010

[8 rows x 9 columns]
```

```
HouseAge: 0 outliers

Dataset Summary:
             MedInc      HouseAge  ...     Longitude   MedHouseVal
count  20640.000000  20640.000000  ...  20640.000000  20640.000000
mean       3.870671     28.639486  ...   -119.569704      2.068558
std        1.899822     12.585558  ...      2.003532      1.153956
min        0.499900      1.000000  ...   -124.350000      0.149990
25%        2.563400     18.000000  ...   -121.800000      1.196000
50%        3.534800     29.000000  ...   -118.490000      1.797000
75%        4.743250     37.000000  ...   -118.010000      2.647250
max       15.000100     52.000000  ...   -114.310000      5.000010

[8 rows x 9 columns]
```

```
AveRooms: 511 outliers

Dataset Summary:
             MedInc      HouseAge  ...     Longitude   MedHouseVal
count  20640.000000  20640.000000  ...  20640.000000  20640.000000
mean       3.870671     28.639486  ...   -119.569704      2.068558
std        1.899822     12.585558  ...      2.003532      1.153956
min        0.499900      1.000000  ...   -124.350000      0.149990
25%        2.563400     18.000000  ...   -121.800000      1.196000
50%        3.534800     29.000000  ...   -118.490000      1.797000
75%        4.743250     37.000000  ...   -118.010000      2.647250
max       15.000100     52.000000  ...   -114.310000      5.000010

[8 rows x 9 columns]
```

```
AveBedrms: 1424 outliers


Dataset Summary:
             MedInc      HouseAge  ...     Longitude   MedHouseVal
count  20640.000000  20640.000000  ...  20640.000000  20640.000000
mean       3.870671     28.639486  ...   -119.569704      2.068558
std        1.899822     12.585558  ...      2.003532      1.153956
min        0.499900      1.000000  ...   -124.350000      0.149990
25%        2.563400     18.000000  ...   -121.800000      1.196000
50%        3.534800     29.000000  ...   -118.490000      1.797000
75%        4.743250     37.000000  ...   -118.010000      2.647250
max       15.000100     52.000000  ...   -114.310000      5.000010

[8 rows x 9 columns]
```

```
Population: 1196 outliers


Dataset Summary:
             MedInc      HouseAge  ...     Longitude   MedHouseVal
count  20640.000000  20640.000000  ...  20640.000000  20640.000000
mean       3.870671     28.639486  ...   -119.569704      2.068558
std        1.899822     12.585558  ...      2.003532      1.153956
min        0.499900      1.000000  ...   -124.350000      0.149990
25%        2.563400     18.000000  ...   -121.800000      1.196000
50%        3.534800     29.000000  ...   -118.490000      1.797000
75%        4.743250     37.000000  ...   -118.010000      2.647250
max       15.000100     52.000000  ...   -114.310000      5.000010

[8 rows x 9 columns]
```

```
AveOccup: 711 outliers


Dataset Summary:
             MedInc      HouseAge  ...     Longitude   MedHouseVal
count  20640.000000  20640.000000  ...  20640.000000  20640.000000
mean       3.870671     28.639486  ...   -119.569704      2.068558
std        1.899822     12.585558  ...      2.003532      1.153956
min        0.499900      1.000000  ...   -124.350000      0.149990
25%        2.563400     18.000000  ...   -121.800000      1.196000
50%        3.534800     29.000000  ...   -118.490000      1.797000
75%        4.743250     37.000000  ...   -118.010000      2.647250
max       15.000100     52.000000  ...   -114.310000      5.000010

[8 rows x 9 columns]
```

```
Latitude: 0 outliers


Dataset Summary:
             MedInc      HouseAge  ...     Longitude   MedHouseVal
count  20640.000000  20640.000000  ...  20640.000000  20640.000000
mean       3.870671     28.639486  ...   -119.569704      2.068558
std        1.899822     12.585558  ...      2.003532      1.153956
min        0.499900      1.000000  ...   -124.350000      0.149990
25%        2.563400     18.000000  ...   -121.800000      1.196000
50%        3.534800     29.000000  ...   -118.490000      1.797000
75%        4.743250     37.000000  ...   -118.010000      2.647250
max       15.000100     52.000000  ...   -114.310000      5.000010

[8 rows x 9 columns]
```

```
Longitude: 0 outliers

Dataset Summary:
             MedInc      HouseAge  ...     Longitude   MedHouseVal
count  20640.000000  20640.000000  ...  20640.000000  20640.000000
mean       3.870671     28.639486  ...   -119.569704      2.068558
std        1.899822     12.585558  ...      2.003532      1.153956
min        0.499900      1.000000  ...   -124.350000      0.149990
25%        2.563400     18.000000  ...   -121.800000      1.196000
50%        3.534800     29.000000  ...   -118.490000      1.797000
75%        4.743250     37.000000  ...   -118.010000      2.647250
max       15.000100     52.000000  ...   -114.310000      5.000010

[8 rows x 9 columns]
```

```
MedHouseVal: 1071 outliers

Dataset Summary:
             MedInc      HouseAge  ...     Longitude   MedHouseVal
count  20640.000000  20640.000000  ...  20640.000000  20640.000000
mean       3.870671     28.639486  ...   -119.569704      2.068558
std        1.899822     12.585558  ...      2.003532      1.153956
min        0.499900      1.000000  ...   -124.350000      0.149990
25%        2.563400     18.000000  ...   -121.800000      1.196000
50%        3.534800     29.000000  ...   -118.490000      1.797000
75%        4.743250     37.000000  ...   -118.010000      2.647250
max       15.000100     52.000000  ...   -114.310000      5.000010

[8 rows x 9 columns]
```

2.Develop a program to compute the correlation matrix to understand the relationships between pairs of features. Visualize the correlation matrix using a heatmap to know which variables have strong positive/negative correlations. Create a pair plot to visualize pairwise relationships between features. Use California Housing dataset.

AIM:

To perform exploratory data analysis (EDA) on the California Housing dataset by computing and visualizing the correlation matrix to understand the relationships between pairs of features. Additionally, the program aims to create pair plots to visualize pairwise relationships between features, providing deeper insights into the dataset.

OBJECTIVES:

- **Load the California Housing dataset**: Utilize the fetch_california_housing function from the sklearn.datasets module to load the dataset into a pandas DataFrame.
- **Compute the correlation matrix**: Calculate the correlation matrix to identify the strength and direction of linear relationships between numerical features in the dataset.
- **Visualize the correlation matrix using a heatmap**: Generate a heatmap of the correlation matrix using the seaborn library, with annotations to highlight strong positive and negative correlations between features.
- **Create a pair plot**: Use the seaborn library to create a pair plot that visualizes pairwise relationships between all numerical features, allowing for a comprehensive analysis of feature interactions.

PROGRAM:

```
# Importing Libraries
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
```

```
#Loading the California Housing Dataset
california_data = fetch_california_housing(as_frame=True)
data = california_data.frame

# Computing the Correlation Matrix
correlation_matrix = data.corr()

#Visualize the correlation matrix using a heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f',
linewidths=0.5)
plt.title('Correlation Matrix of California Housing Features')
plt.show()

#Create a pair plot to visualize pairwise relationships
sns.pairplot(data, diag_kind='kde', plot_kws={'alpha': 0.5})
plt.suptitle('Pair Plot of California Housing Features', y=1.02)
plt.show()
```

EXPLANATION:

Importing Libraries:

- **pandas** is used for handling tabular data (dataframes).
- **seaborn**: A visualization library based on matplotlib that provides a high-level interface for drawing attractive statistical graphics.
- **matplotlib.pyplot** is used for plotting graphs.
- **fetch_california_housing** from **sklearn.datasets** loads the California Housing dataset.

Loading the California Housing Dataset:

- **fetch_california_housing(as_frame=True):** Loads the California Housing dataset as a pandas DataFrame.

- **data = california_data.frame:** Extracts the dataset as a DataFrame for easier manipulation and visualization.

Computing the Correlation Matrix:

- **correlation_matrix = data.corr() :** .corr() computes the correlation matrix, which shows the statistical relationship between numerical features.
- **data.corr()**: Computes the pairwise correlation of columns, excluding NA/null values. The result is a DataFrame showing the correlation coefficients between the features.
- The correlation values range from **-1 to 1**, where:
  - **1** indicates a strong positive correlation.
  - **-1** indicates a strong negative correlation.
  - **0** indicates no correlation.

Visualizing the Correlation Matrix using a Heatmap:

- **plt.figure(figsize=(10, 8))** sets the figure size as 10x8 inches for better readability.
- **sns.heatmap():** Plots the heatmap to visualize correlation
  - **correlation_matrix:** The data to plot.
  - **annot=True**: Annotates each cell with its value i.e., displays correlation values on the heatmap
  - **cmap='coolwarm'**: Uses the 'coolwarm' colormap to indicate correlation strength
  - **fmt='.2f'**: Formats the annotations(numbers) to two decimal places.
  - **linewidths=0.5**: Sets the width of the lines that divide the cells.
- **plt.title():** Adds a title to the heatmap.
- **plt.show():** Displays the plot.

Creating a Pair Plot to Visualize Pairwise Relationships:

sns.pairplot(data): Creates pairwise scatter plots for all numerical features in the dataset.
data: The dataset to plot.
diag_kind='kde': Uses Kernel Density Estimation (KDE) for diagonal plots instead of histograms to show data distribution.
plot_kws={'alpha': 0.5}: Sets the transparency level of the plots to 50% for better visibility.

plt.suptitle(): Adds a title to the pair plot, adjusting its position (y=1.02).

plt.show(): Displays the pair plot.

Kernel Density Estimate (KDE):

Kernel Density Estimation (KDE) is a technique used to estimate the probability density function (PDF) of a continuous random variable. It smooths the distribution of data points by placing a kernel (a weighted function) over each data point and summing them to create a smooth curve.

Why Use KDE?

1. **Smooth Representation:** Unlike histograms, which depend on bin sizes, KDE provides a smooth and continuous estimate of data distribution.
2. **Better Visualization:** Helps in understanding the underlying distribution of a dataset without being affected by bin edges.
3. **Identifying Patterns:** Useful for detecting multiple peaks (modes) in data, indicating multimodal distributions.

Probability Density Function (PDF):

A **Probability Density Function (PDF)** is a function that describes the likelihood of a continuous random variable taking on a particular value. The PDF does not give the probability of a specific value but instead provides the probability density, which must be integrated over an interval to obtain actual probabilities.

Properties of a PDF:

1. **Non-Negativity:** $f(x) \geq 0$ for all values of x.
2. **Total Probability is 1:** The area under the PDF curve must equal 1.

$$\int_{-\infty}^{\infty} f(x)dx = 1$$

3. **Probability of a Single Value is Zero:** Since a continuous variable can take infinitely many values, the probability of any exact value is **zero**:

$$P(X = x) = 0$$

Instead, probabilities are calculated over intervals:

$$P(a \leq X \leq b) = \int_a^b f(x)dx$$

Example: Normal (Gaussian) PDF

The **Normal distribution** (bell curve) is one of the most common PDFs, defined as:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Where:

- μ is the **mean** (center of the distribution),
- σ is the **standard deviation** (spread of the data),
- e is Euler's number (≈2.718).

OUTPUT:

Figure1 shows the Correlation Matrix Heatmap of California Housing Features. The heatmap visualizes the correlation coefficients between the different housing-related variables in the California Housing dataset

Correlation values range from -1 to 1:

- **1 (Red)** → Strong positive correlation
- **0 (Neutral, Light color)** → No correlation
- **-1 (Blue)** → Strong negative correlation

The diagonal elements are all **1.00** because a variable is always perfectly correlated with itself.

Key Observations from the Heatmap:

Strongest Positive Correlations:

- **Median Income (MedInc) vs. Median House Value (MedHouseVal)**
  MedInc (Median Income) and MedHouseVal (Median House Value) have a strong positive correlation of 0.69. This means that as median income increases, median house values tend to increase as well.
- **Average Rooms (AveRooms) vs. Average Bedrooms (AveBedrms)**
  AveRooms (Average Rooms) and AveBedrms (Average Bedrooms) have a very high positive correlation of 0.85, suggesting that houses with more rooms generally also have more bedrooms.
- **HouseAge vs Median House Value (MedHouseVal)**
  HouseAge and MedHouseVal show a weak positive correlation of 0.11, indicating that there is a slight tendency for older houses to have higher values.

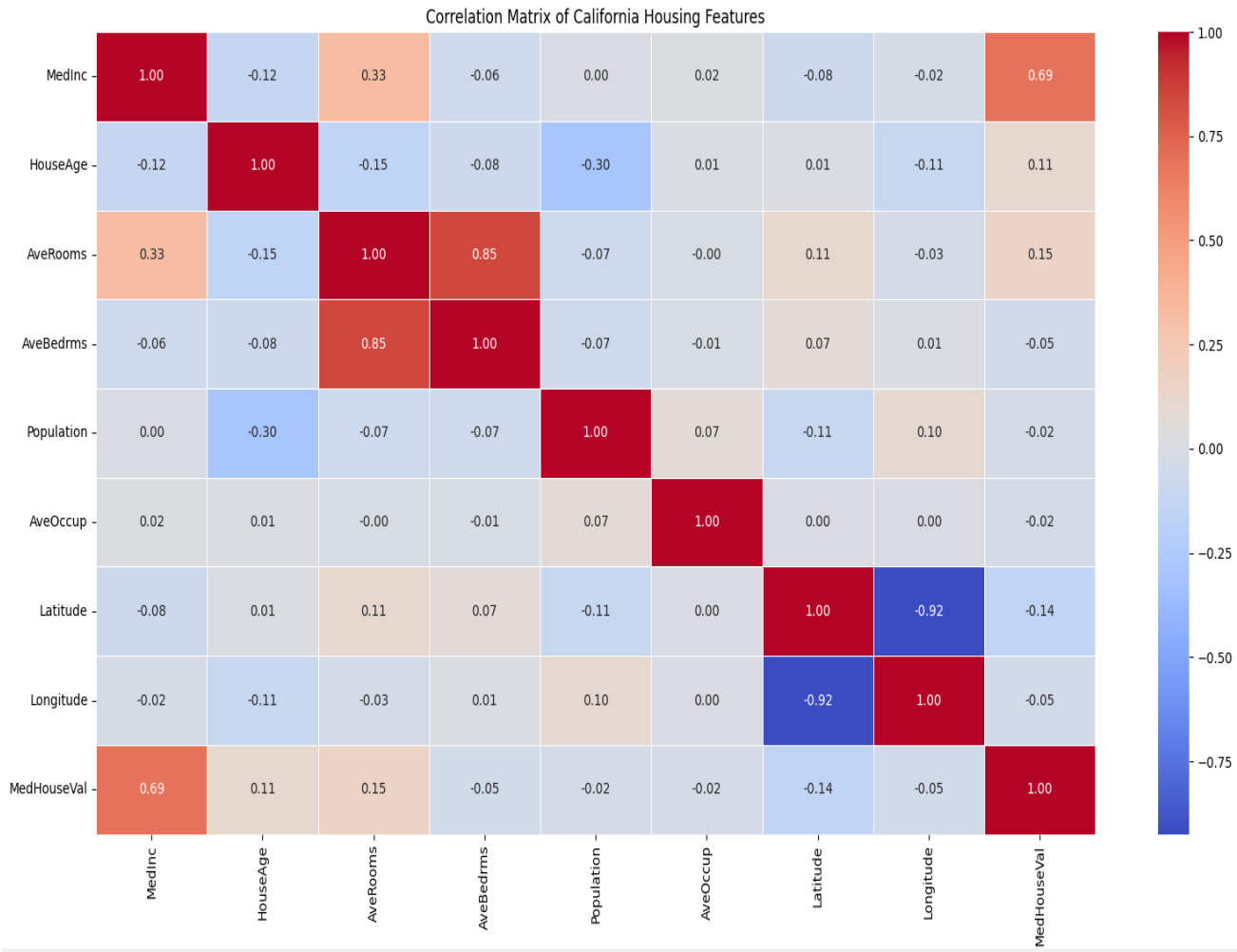


Figure2.1: Correlation matrix of California Housing Features

Strongest Negative Correlations:

- **Latitude vs. Longitude**

  Latitude and Longitude have a strong negative correlation of -0.92. This implies that as one increases, the other decreases (likely due to the dataset covering a specific region in California).

- **Population vs. House Age**

  Population and HouseAge have a moderate negative correlation of -0.30, indicating that areas with older houses tend to have lower populations.

Feature Selection: MedInc has a strong impact on house prices (MedHouseVal), so it is an important feature for predictive modeling.

Multicollinearity Consideration: AveRooms and AveBedrms are highly correlated, meaning one may be redundant in a regression model.

Geographical Trends: The strong negative correlation between Latitude and Longitude suggests location-based patterns in housing prices.

This analysis helps in understanding feature relationships, guiding **feature selection**, **data preprocessing**, and **model building** for price prediction in the California housing dataset.

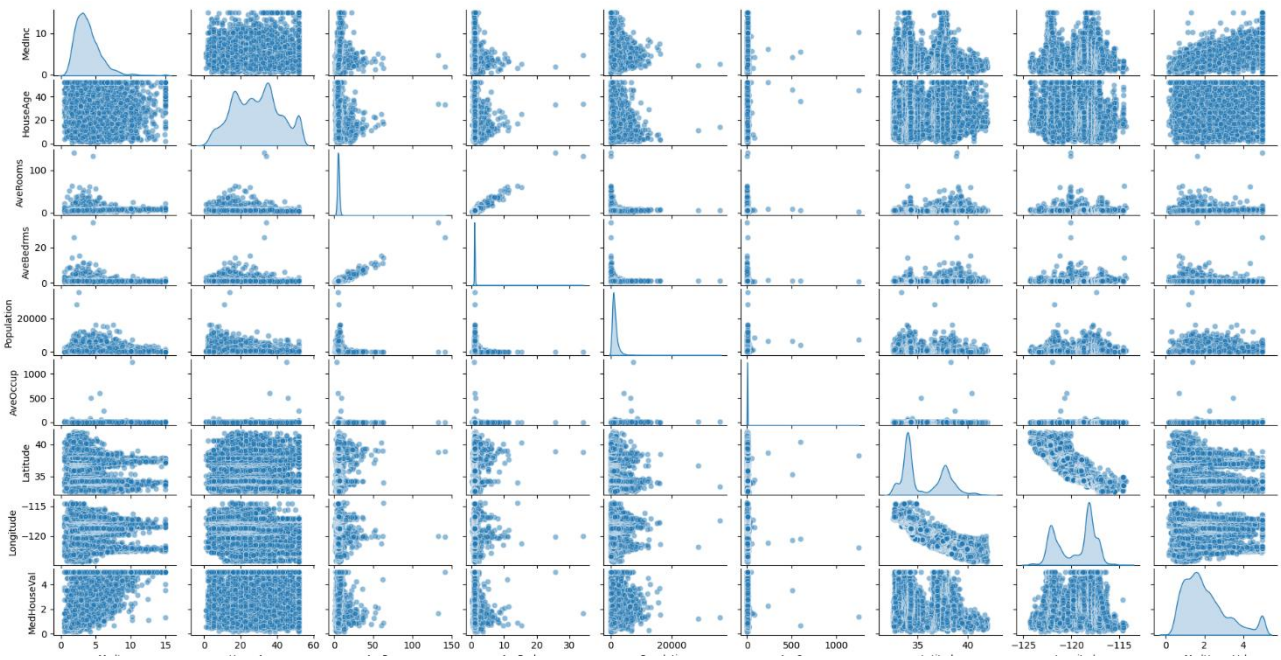


Figure2.2: Pair Plot of California Housing Features

The pair plot shown in Figure2.2 visualizes relationships between multiple variables in the California Housing dataset. It includes scatter plots for each pair of features and histograms or KDE plots on the diagonal.

Key Observations:

Distributions (Diagonal Plots):

- **MedInc (Median Income):** Positively skewed distribution, indicating more households have a lower median income.
- **HouseAge (House Age):** Shows a bimodal distribution, suggesting two groups of houses - older and newer.
- **AveRooms (Average Rooms):** Right-skewed distribution with some outliers indicating houses with exceptionally high numbers of rooms.
- **AveBedrms (Average Bedrooms):** Also right-skewed with fewer extreme outliers.
- **Population:** Distribution indicates most locations have relatively lower populations, with a few highly populated areas.
- **AveOccup (Average Occupancy):** Highly skewed with a few outliers indicating locations with very high average occupancy.
- **Latitude and Longitude:** Uniform distribution, reflecting geographic spread across California.
- **MedHouseVal (Median House Value):** Right-skewed distribution with many houses having lower values, but a few with significantly higher values.

Relationships (Scatter Plots):

- **MedInc vs. MedHouseVal:** Positive correlation, as higher median incomes are associated with higher median house values.
- **AveRooms vs. AveBedrms:** Strong positive correlation, as houses with more rooms generally have more bedrooms.

- **HouseAge vs. Population:** Negative correlation, suggesting older houses tend to be in less populated areas.
- **Latitude vs. Longitude:** Negative correlation, reflecting the geographic orientation of the dataset.

Outliers and Trends:

- Outliers are present in **AveRooms**, **AveBedrms**, **AveOccup**, and **Population**. These outliers may represent unique locations or housing situations that are significantly different from the norm.
- **MedInc (Median Income) and MedHouseVal (Median House Value)** show a clear positive trend, indicating that areas with higher incomes tend to have higher house values.

The pair plot helps in **understanding relationships, feature importance, and detecting outliers** before using the dataset for predictive modeling. The observed trends, especially between MedInc and MedHouseVal, confirm that income levels play a major role in house prices.

3.Develop a program to implement Principal Component Analysis (PCA) for reducing the dimensionality of the Iris dataset from 4 features to 2.

AIM:

To visualize the Iris dataset using Principal Component Analysis (PCA) to reduce its dimensionality from 4 to 2 dimensions. The reduced data is then plotted to observe the separation of different Iris species.

OBJECTIVES:

- **Load and Explore Data:** Load the Iris dataset and convert it to a panda DataFrame for easier visualization and manipulation.
- **Dimensionality Reduction:** Apply PCA to reduce the dimensionality of the dataset from 4 features to 2 principal components.
- **Data Transformation:** Create a new DataFrame containing the reduced data along with the corresponding labels.
- **Visualization:** Plot the reduced data on a 2D plane, with each point representing a sample from the Iris dataset and colors indicating different Iris species.
- **Analysis and Interpretation:** Analyze the plot to determine the separation and clustering of the different Iris species based on the principal components.

PROGRAM:

```
# Importing Libraries
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# Load the Iris dataset
iris = load_iris()
data = iris.data
labels = iris.target
```

```
label_names = iris.target_names
# Convert to a DataFrame for better visualization
iris_df = pd.DataFrame(data, columns=iris.feature_names)
# Perform PCA to reduce dimensionality to 2
pca = PCA(n_components=2)
data_reduced = pca.fit_transform(data)

# Create a DataFrame for the reduced data
reduced_df = pd.DataFrame(data_reduced, columns=['Principal Component 1',
'Principal Component 2'])
reduced_df['Label'] = labels

# Plot the reduced data
plt.figure(figsize=(8, 6))
colors = ['r', 'g', 'b']
for i, label in enumerate(np.unique(labels)):
        plt.scatter(
        reduced_df[reduced_df['Label'] == label] ['Principal Component 1'],
        reduced_df[reduced_df['Label'] == label] ['Principal Component 2'],
        label=label_names[label],
        color=colors[i]
        )
plt.title('PCA on Iris Dataset')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend()
plt.grid()
plt.show()
```

EXPLANATION:

Importing Libraries

numpy (np): A library for numerical operations.

pandas (pd): A library for data manipulation and analysis.

sklearn.datasets.load_iris: Imports the Iris dataset.

sklearn.datasets.load_iris: imports the Iris dataset.

sklearn.decomposition.PCA: A class to perform Principal Component Analysis (PCA). Imports Principal Component Analysis (PCA) for dimensionality reduction.

matplotlib.pyplot (plt): A library for plotting graphs and visualizations.

Load the Iris dataset

iris = load_iris() loads the Iris dataset, which contains 150 samples of flowers, each having 4 features into the variable iris.

iris.data: Extracts feature values (sepal length, sepal width, petal length, petal width) from the dataset.

iris.target: Extracts the target labels (species - 0, 1, or 2), representing three flower species from the dataset.

iris.target_names: Extracts the names of the species ('setosa', 'versicolor', 'virginica') from the dataset.

Convert data into a DataFrame for better visualization

iris_df = pd.DataFrame(data, columns=iris.feature_names)

- Converts the feature data into a Pandas DataFrame.
- Assigns column names based on iris.feature_names (sepal length (cm), sepal width (cm), etc.).

Perform PCA to reduce dimensionality to 2

PCA(n_components=2): Creates a PCA object to reduce the data to 2 principal components.

pca.fit_transform(data): Applies PCA on the original dataset and transforms it into two principal components.

Create a DataFrame for the reduced data

reduced_df = pd.DataFrame(data_reduced, columns=['Principal Component 1', 'Principal Component 2']) reduced_df['Label'] = labels

- Creates a DataFrame with the reduced data, naming the columns 'Principal Component 1' and 'Principal Component 2'.

- Adds the Label column containing the original class labels to the DataFrame.

Plot the reduced data

plt.figure(figsize=(8, 6)) creates a figure with a size of 8x6 inches.

colors = ['r', 'g', 'b'] specifies colors for the three species (red, green, blue).

for i, label in enumerate(np.unique(labels)): iterates through the unique species labels (0, 1, 2).

plt.scatter(...):

- Filters **reduced_df** to select points belonging to the current label.
- Plots those points on a scatter plot using **plt.scatter().**
- Assigns a color and label to each species.

Customize and show the plot

plt.title('PCA on Iris Dataset') sets the title of the plot.

plt.xlabel('Principal Component 1') labels the x-axis.

plt.ylabel('Principal Component 2') labels the y-axis.

plt.legend() displays a legend indicating which color represents which species.

plt.grid() adds a grid to the plot.

plt.show() displays the plot.

OUTPUT:

Figure3.1 is a visual representation of the Iris dataset after applying Principal Component Analysis (PCA) to reduce its dimensionality from 4 to 2 components.

Data Transformation using PCA

- The **original dataset** consists of **4 features**:
  - Sepal Length
  - Sepal Width
  - Petal Length
  - Petal Width
- Since it's difficult to visualize **4D data**, PCA reduces it to **2 principal components** that **capture most of the variance** in the data.

Understanding the Clusters

The scatter plot shows three distinct clusters, each corresponding to one of the three **Iris species**:

- **Red points (Setosa)**
  - These form a well-separated cluster on the left side of the plot.
  - This suggests that the Setosa species is **easily distinguishable** from the other two.
- **Green points (Versicolor)**
  - Located mostly in the center of the plot.
  - There is **some overlap** with the Virginica species, indicating similarity in certain features.
- **Blue points (Virginica)**
  - Spread more towards the right side of the plot.
  - Overlaps slightly with Versicolor, suggesting some similarity.

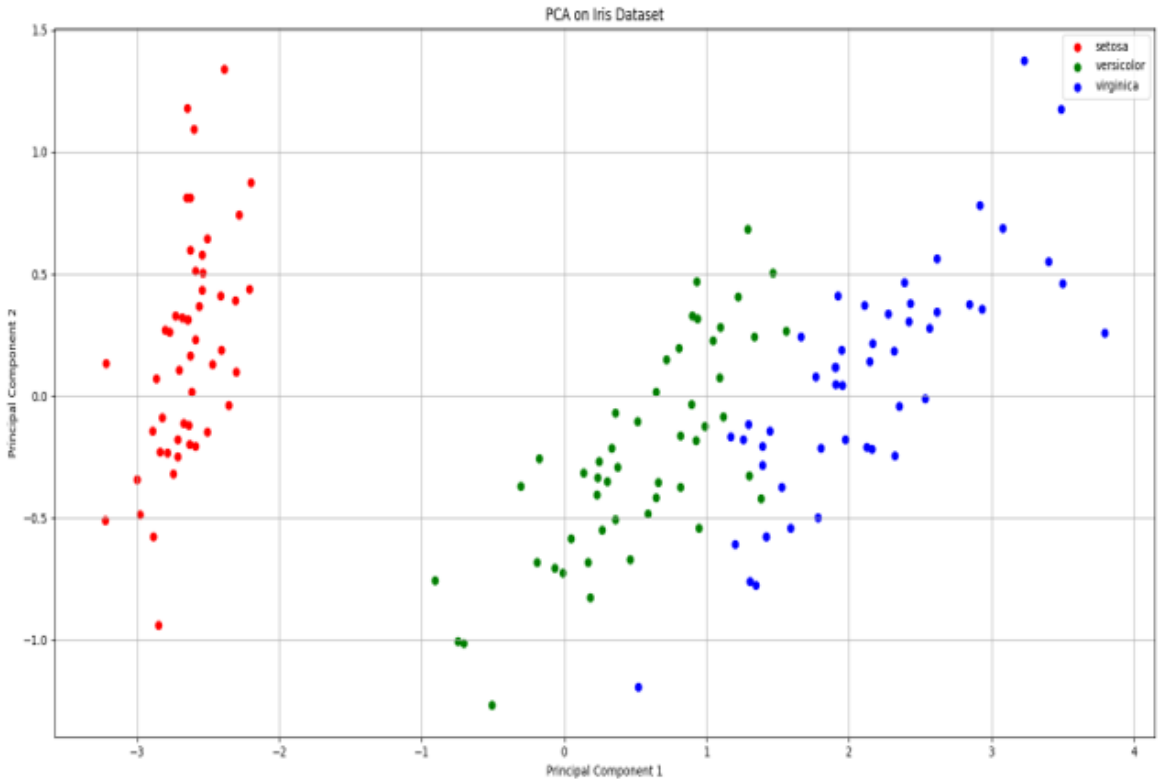


Figure3.1: PCA Scatter Plot of the Iris Dataset

Observations & Insights

- Setosa is linearly separable

  - In the original dataset, Setosa had distinct petal characteristics, making it easier to classify.
- Versicolor and Virginica have some overlap
  - The overlap suggests that these species share some feature similarities, making classification a bit more challenging.
- Principal Component 1 (PC1) explains most of the variance
  - PC1 is the horizontal axis, which spreads the data across a wide range.
  - It is likely capturing the most significant feature variations (probably petal length and width).
- Principal Component 2 (PC2) captures lesser variance
  - PC2 is the vertical axis, contributing to the spread but with less impact compared to PC1.

Conclusion

- PCA successfully reduces the dataset to 2D while retaining key patterns.
- The clusters align with the species labels, making visualization effective.
- Setosa is distinctly separate, while Versicolor and Virginica show some overlap.

This visualization is useful for data exploration, helping to understand relationships between species before applying machine learning classification techniques.

4. For a given set of training data examples stored in a .CSV file, implement and demonstrate the Find-S algorithm to output a description of the set of all hypotheses consistent with the training examples.

AIM:

To implement the Find-S algorithm, which is a simple machine learning algorithm used for learning the most specific hypothesis that fits all the positive examples in a given training dataset.

OBJECTIVES:

- **Load Training Data:** Import the training data from a CSV file using the Pandas library to create a DataFrame.
- **Initialize Hypothesis:** Set the initial hypothesis to the most general hypothesis (i.e., all attributes are set to '?').
- **Iterate Through Examples:** Iterate through each example in the training data to refine the hypothesis.
- **Update Hypothesis:** For each positive example (where the class label is 'Yes'), update the hypothesis by keeping only the consistent attribute values and generalizing others.
- **Output Hypothesis:** After processing all positive examples, output the final hypothesis, which represents the most specific description of the positive examples.

PROGRAM:

```
# Importing the Pandas Library
import pandas as pd

# Define the Function
def find_s_algorithm(file_path):

# Reading the CSV File
    data = pd.read_csv(file_path)
    print("Training data:")
```

```
    print(data)
# Extracting Attributes and Class Labels
    attributes = data.columns[:-1]
    class_label = data.columns[-1]
# Initializing the Hypothesis
    hypothesis = ['?' for _ in attributes]

    for index, row in data.iterrows():
        if row[class_label] == 'Yes':
            #Updating the Hypothesis
            for i, value in enumerate(row[attributes]):
                if hypothesis[i] == '?' or hypothesis[i] == value:
                    hypothesis[i] = value
                else:
                    hypothesis[i] = '?'
    return hypothesis
#Specifying the File Path and Running the Algorithm
file_path = r'C:\Users\Nalini\Downloads\training_data.csv'
hypothesis = find_s_algorithm(file_path)
print("\nThe final hypothesis is:", hypothesis)
```

EXPLANATION:

import pandas as pd imports the pandas library, which is essential for handling and manipulating the dataset.

def find_s_algorithm(file_path): defines the find_s_algorithm function, which takes a single argument file_path representing the path to the training data CSV file.

Reading the CSV File

```
data = pd.read_csv(file_path)
print("Training data:")
print(data)
```

The block reads the CSV file into a DataFrame named data and prints the training data. The pd.read_csv(file_path) function is used to read the CSV file.

Extracting Attributes and Class Labels

attributes = data.columns[:-1]

class_label = data.columns[-1]

Here, attributes are all column names except the last one, which is assumed to be the class label. class_label stores the name of the last column.

Initializing the Hypothesis

hypothesis = ['?' for _ in attributes] : The initial hypothesis is set to the most general hypothesis, where all attributes are set to '?'.

Iterating Through the Training Data

for index, row in data.iterrows(): starts a loop that iterates through each row of the training data. data.iterrows() returns an iterator that produces index and row pairs.

Check for Positive Examples

if row[class_label] == 'Yes': This condition checks if the class label of the current row is 'Yes'. Only positive examples are used to update the hypothesis.

Updating the Hypothesis

```
for i, value in enumerate(row[attributes]):
    if hypothesis[i] == '?' or hypothesis[i] == value:
        hypothesis[i] = value
    else:
        hypothesis[i] = '?'
```

This nested loop iterates through each attribute value in the current row. The hypothesis is updated as follows:

- If the hypothesis at index i is '?' or matches the current attribute value, it is set to the current value.
- Otherwise, it is generalized to '?'.

Returning the Final Hypothesis

return hypothesis returns the final hypothesis after processing all positive examples.

Specifying the File Path and Running the Algorithm

file_path = r'C:\Users\Nalini\Downloads\training_data.csv'

hypothesis = find_s_algorithm(file_path)

print("\nThe final hypothesis is:", hypothesis)

These lines specify the file path to the training data, call the find_s_algorithm function, and print the final hypothesis.

OUTPUT:

```
C:\Users\Nalini\PycharmProjects\pythonProject1\.venv\Scripts\python.exe C:\Users\Nalini\PycharmProjects\pythonProject1\4.py
Training data:
     Outlook Temperature Humidity    Wind PlayTennis
0      Sunny         Hot     High    Weak         No
1      Sunny         Hot     High  Strong         No
2   Overcast         Hot     High    Weak        Yes
3       Rain        Mild     High    Weak        Yes
4       Rain        Cool   Normal    Weak        Yes
5       Rain        Cool   Normal  Strong         No
6   Overcast        Cool   Normal  Strong        Yes
7      Sunny        Mild     High    Weak         No
8      Sunny        Cool   Normal    Weak        Yes
9       Rain        Mild   Normal    Weak        Yes
10     Sunny        Mild   Normal  Strong        Yes
11  Overcast        Mild     High  Strong        Yes
12  Overcast         Hot   Normal    Weak        Yes
13      Rain        Mild     High  Strong         No

The final hypothesis is: ['Overcast', '?', 'Normal', '?']

Process finished with exit code 0
```

Training Data

The training data consists of various weather conditions and whether or not tennis was played (PlayTennis column). The attributes are Outlook, Temperature, Humidity, and Wind.

Find-S Algorithm Explanation

The Find-S algorithm aims to find the most specific hypothesis that fits all positive examples (i.e., where PlayTennis is 'Yes').

Positive Examples

Here are the positive examples from the training data (rows where PlayTennis is 'Yes'):

Outlook	Temperature	Humidity	Wind	PlayTennis
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Overcast	Cool	Normal	Strong	Yes
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes

Hypothesis Updates

1. **Initial Hypothesis:** ['?', '?', '?', '?']
2. **Example 1 (Overcast, Hot, High, Weak):** ['Overcast', 'Hot', 'High', 'Weak']
3. **Example 2 (Rain, Mild, High, Weak):** ['?', '?', 'High', 'Weak']
4. **Example 3 (Rain, Cool, Normal, Weak):** ['?', '?', '?', 'Weak']
5. **Example 4 (Overcast, Cool, Normal, Strong):** ['Overcast', '?', '?', '?']
6. **Example 5 (Sunny, Cool, Normal, Weak):** ['?', '?', '?', '?']
7. **Example 6 (Rain, Mild, Normal, Weak):** ['?', '?', 'Normal', '?']
8. **Example 7 (Sunny, Mild, Normal, Strong):** ['?', '?', '?', '?']
9. **Example 8 (Overcast, Mild, High, Strong):** ['Overcast', '?', '?', '?']
10. **Example 9 (Overcast, Hot, Normal, Weak):** ['Overcast', '?', 'Normal', '?']

Final Hypothesis

The final hypothesis is: ['Overcast', '?', 'Normal', '?']

- **Overcast:** The attribute Outlook is consistently 'Overcast' in some positive examples, and no conflicting values were found for this attribute that generalized to '?'.
- **Temperature:** The attribute Temperature varies across positive examples ('Hot', 'Cool', 'Mild'), so it is generalized to '?'.
- **Humidity:** The attribute Humidity has consistent values ('High', 'Normal') in positive examples where Outlook is 'Overcast', so it is set to 'Normal'.
- **Wind:** The attribute Wind varies across positive examples ('Weak', 'Strong'), so it is generalized to '?'.

The Find-S algorithm provides the most specific hypothesis that fits all positive examples in the training data. In this case, the final hypothesis suggests that if the Outlook is 'Overcast' and Humidity is 'Normal', then PlayTennis is 'Yes', regardless of Temperature and Wind.

5. Develop a program to implement k-Nearest Neighbour algorithm to classify the randomly generated 100 values of *x* in the range of [0,1]. Perform the following based on dataset generated.

1. Label the first 50 points {*x1*,……,*x50*} as follows: if (xi ≤ 0.5), then *xi* ε Class1, else *xi* ε Class1

2. Classify the remaining points, *x51*,……,*x100* using KNN. Perform this for *k=1,2,3,4,5,20,30*

AIM:

To implement a k-Nearest Neighbors (k-NN) classifier to classify a set of data points based on their Euclidean distance to the nearest neighbors in the training dataset. The program explores the effect of different values of kk on the classification accuracy and performance.

OBJECTIVES:

- **Random Data Generation:**
  Generate 100 random data points uniformly distributed between 0 and 1.
- **Data Labeling:**
  Label the first 50 data points based on the rule: x <= 0.5 -> Class1, x > 0.5 -> Class2.
- **Define Euclidean Distance Function:**
  Create a function to calculate the Euclidean distance between two data points.
- **Implement k-NN Classifier:**
  Develop a function to classify a given test point based on its k nearest neighbors from the training data.
- **Split Data:**
  Divide the generated data into training and testing datasets. Use the first 50 points as the training set and the remaining 50 points as the testing set.
- Classification with Various k Values:
  Use the k-NN classifier to classify the testing dataset for different values of k (e.g., 1, 2, 3, 4, 5, 20, 30).
- **Output Results:**
  Print the classification results for each value of k, showing which class each test data point is assigned to.

PROGRAM:

```
#Import Libraries:
import numpy as np
import matplotlib.pyplot as plt
from collections import Counter

# Generate random data
data = np.random.rand(100)

# Label the first 50 data points
labels = ["Class1" if x <= 0.5 else "Class2" for x in data[:50]]

# Define Euclidean distance function
def euclidean_distance(x1, x2):
    return np.sqrt((x1 - x2)**2)

# Define k-NN classifier function
def knn_classifier(train_data, train_labels, test_point, k):
    distances = [(euclidean_distance(test_point, train_data[i]), train_labels[i]) for i in
range(len(train_data))]
    distances.sort(key=lambda x: x[0])
    k_nearest_neighbors = distances[:k]
    k_nearest_labels = [label for _, label in k_nearest_neighbors]
    return Counter(k_nearest_labels).most_common(1)[0][0]

# Split data into training and testing datasets
train_data = data[:50]
train_labels = labels
test_data = data[50:]

k_values = [1, 2, 3, 4, 5, 20, 30]
print("--- k-Nearest Neighbors Classification ---")
```

```
print("Training dataset: First 50 points labeled based on the rule (x <= 0.5 -> Class1, x > 0.5 -
> Class2)")
print("Testing dataset: Remaining 50 points to be classified\n")

# Run k-NN classification for different k values
results = {}
for k in k_values:
    print(f"Results for k = {k}:")
    classified_labels = [knn_classifier(train_data, train_labels, test_point, k) for test_point in
test_data]
    results[k] = classified_labels
    for i, label in enumerate(classified_labels, start=51):
        print(f"Point x{i} (value: {test_data[i - 51]:.4f}) is classified as {label}")
    print("\n")
print("Classification complete.\n")

# Plot the classification results for each k value
for k in k_values:
    classified_labels = results[k]
    class1_points = [test_data[i] for i in range(len(test_data)) if classified_labels[i] ==
"Class1"]
    class2_points = [test_data[i] for i in range(len(test_data)) if classified_labels[i] ==
"Class2"]
    plt.figure(figsize=(10, 6))
    plt.scatter(train_data, [0] * len(train_data), c=["blue" if label == "Class1" else "red" for
label in train_labels],
                label="Training Data", marker="o")
    plt.scatter(class1_points, [1] * len(class1_points), c="blue", label="Class1 (Test)",
marker="x")
    plt.scatter(class2_points, [1] * len(class2_points), c="red", label="Class2 (Test)",
marker="x")
    plt.title(f"k-NN Classification Results for k = {k}")
    plt.xlabel("Data Points")
```

```
plt.ylabel("Classification Level")
plt.legend()
plt.grid(True)
plt.show()
```

EXPLANATION:

Import Libraries:

numpy: For numerical operations**.**

matplotlib.pyplot: For plotting, although not used in this code.

collections.Counter: To count the frequency of elements.

Generate Random Data:

data = np.random.rand(100) creates an array of 100 random values uniformly distributed between 0 and 1.

Label Data Points:

labels = ["Class1" if x <= 0.5 else "Class2" for x in data[:50]]

Label the first 50 data points as "Class1" if the value is less than or equal to 0.5, otherwise label them as "Class2".

Define Euclidean Distance Function:

```
def euclidean_distance(x1, x2):
    return np.sqrt((x1 - x2)**2)
```

calculates the Euclidean distance between two points x1 and x2.

Define k-NN classifier function:

def knn_classifier(train_data, train_labels, test_point, k): classifies a new test point using the k-nearest neighbors (k-NN) algorithm.

distances = [(euclidean_distance(test_point, train_data[i]), train_labels[i]) for i in range(len(train_data))] computes the distance between the test point and every training data point. Stores distances along with corresponding labels.

distances.sort(key=lambda x: x[0]) sorts distances in ascending order (smallest distance first).

k_nearest_neighbors = distances[:k] selects the k closest training points.

k_nearest_labels = [label for _, label in k_nearest_neighbors] extracts the labels of these k nearest points.

return Counter(k_nearest_labels).most_common(1)[0][0] counts occurrences of each label. Returns the most frequent label among the k neighbors.

Split data into training and testing sets:

train_data = data[:50]

train_labels = labels

test_data = data[50:]

- First 50 data points are used for training (with labels).
- Last 50 data points are used for testing (without labels).

Define k values:

k_values = [1, 2, 3, 4, 5, 20, 30] - different values of k to test.

Print dataset details:

print("--- k-Nearest Neighbors Classification ---")

print("Training dataset: First 50 points labeled based on the rule (x <= 0.5 -> Class1, x > 0.5 -> Class2)")

print("Testing dataset: Remaining 50 points to be classified\n")

Provides information about training and testing datasets.

Run k-NN classification for different values of k:

results = {}

for k in k_values: results dictionary stores classification results for each k.

print(f"Results for k = {k}:") prints the current value of k.

classified_labels = [knn_classifier(train_data, train_labels, test_point, k) for test_point in test_data] classifies each test data point using the knn_classifier function.

results[k] = classified_labels stores the classified labels in the results dictionary.

for i, label in enumerate(classified_labels, start=51):

print(f"Point x{i} (value: {test_data[i - 51]:.4f}) is classified as {label}")

prints classification results, showing test point index, value, and assigned class.

Plot the classification results:

for k in k_values: loops through each k to visualize classification results.

classified_labels = results[k]

class1_points = [test_data[i] for i in range(len(test_data)) if classified_labels[i] == "Class1"]

class2_points = [test_data[i] for i in range(len(test_data)) if classified_labels[i] == "Class2"] separates test points into Class1 and Class2 based on classification.

plt.figure(figsize=(10, 6)) creates a new plot with size 10x6 inches.

plt.scatter(train_data, [0] * len(train_data), c=["blue" if label == "Class1" else "red" for label in train_labels],

label="Training Data", marker="o") plots training data at y=0 using circles (o). Blue = Class1, Red = Class2.

plt.scatter(class1_points, [1] * len(class1_points), c="blue", label="Class1 (Test)", marker="x")

plt.scatter(class2_points, [1] * len(class2_points), c="red", label="Class2 (Test)", marker="x")

Plots classified test data at y=1:

- Blue X = Class1
- Red X = Class2

```
plt.title(f"k-NN Classification Results for k = {k}")
plt.xlabel("Data Points")
plt.ylabel("Classification Level")
plt.legend()
plt.grid(True)
plt.show()
```

Adds title, labels, legend, and grid. Displays the plot.

OUTPUT:

```
--- k-Nearest Neighbors Classification ---
Training dataset: First 50 points labeled based on the rule (x <= 0.5 -> Class1, x > 0.5 -> Class2)
Testing dataset: Remaining 50 points to be classified
```

The following output graphs represent k-Nearest Neighbors (k-NN) classification results for different values of kkk using a simple 1D dataset. Below is an analysis of the graphs:

General Layout:

- The **blue circles** at the bottom (classification level = 0) represent the training data.

- The **blue and red crosses** at the top (classification level = 1) represent test data classified as **Class1** and **Class2**, respectively.
- The horizontal axis represents the data points (values between 0 and 1), and the classification is performed using k-NN.

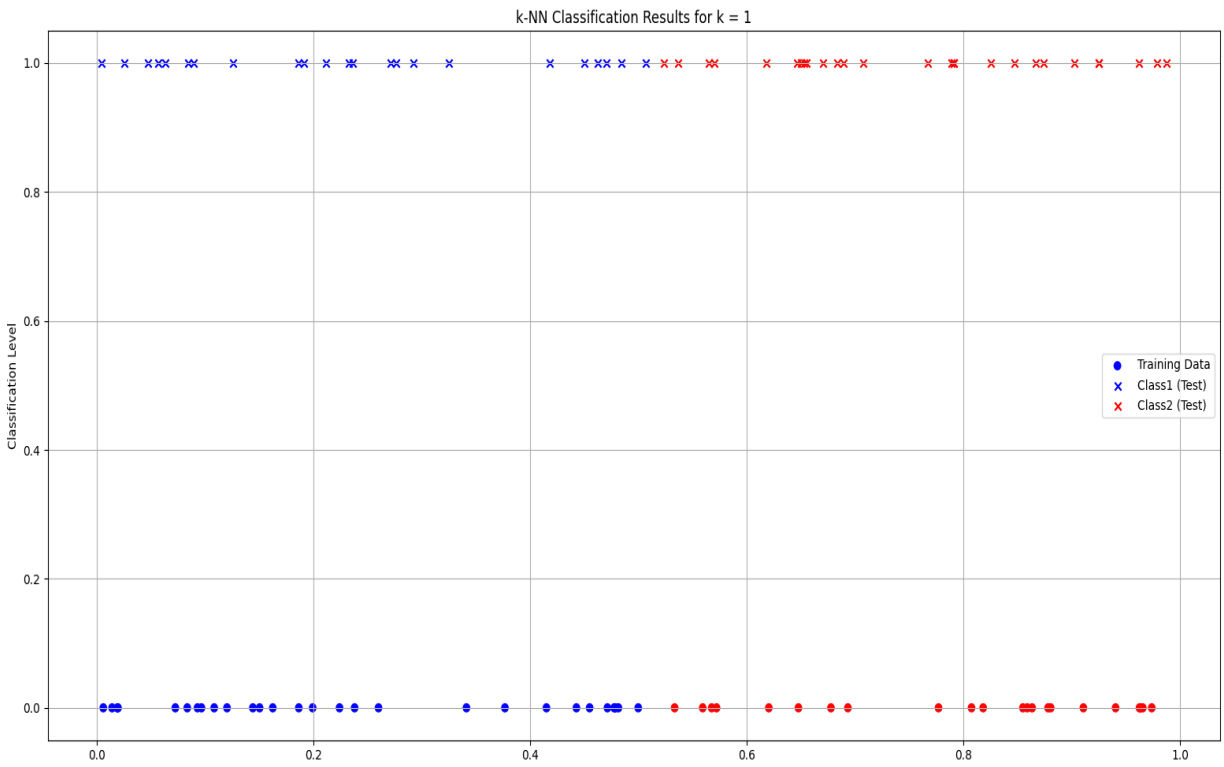
Effect of Different kkk Values:

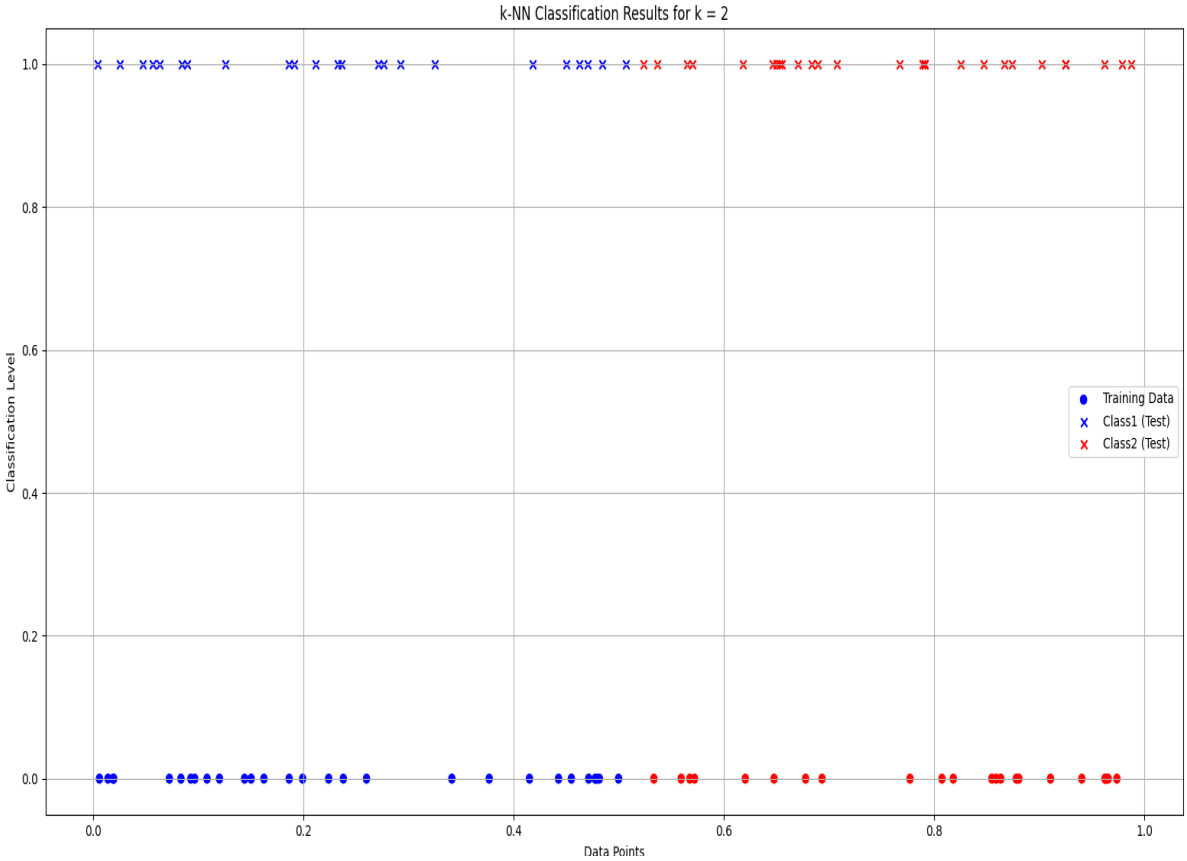
- **1k=1**:
  - The classification is very sensitive to noise.
  - Each test point is classified based on its nearest neighbor, leading to **potential misclassification due to local outliers**.
  - This results in a highly variable decision boundary.
- **k=2**:
  - There is some smoothing compared to k=1, but decisions are still largely **based on very local patterns**.
- **k=3and k=4**:
  - As k increases, the model becomes **more stable**, reducing the risk of overfitting.
  - The decision boundary between **Class1 (blue crosses) and Class2 (red crosses)** becomes more distinct.
- **k=5 and k=20:**
  - The classification is significantly smoothed, meaning the majority class has a stronger influence on classification.
  - The model starts to resemble a more generalized decision rule, reducing misclassification at the cost of losing sensitivity to finer details.
  - At k=20, the decision boundary is nearly deterministic based on the majority class, meaning classification is almost entirely dictated by global trends rather than local details.

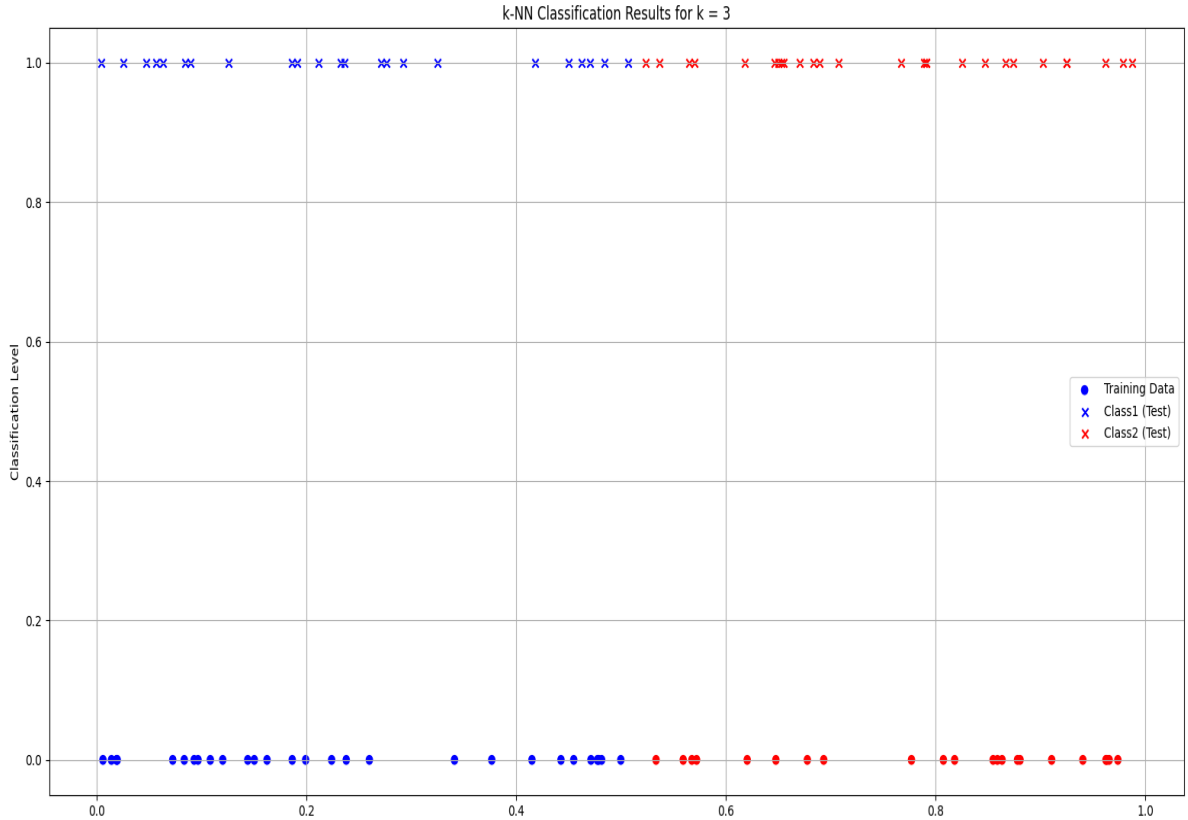
Key Takeaways:

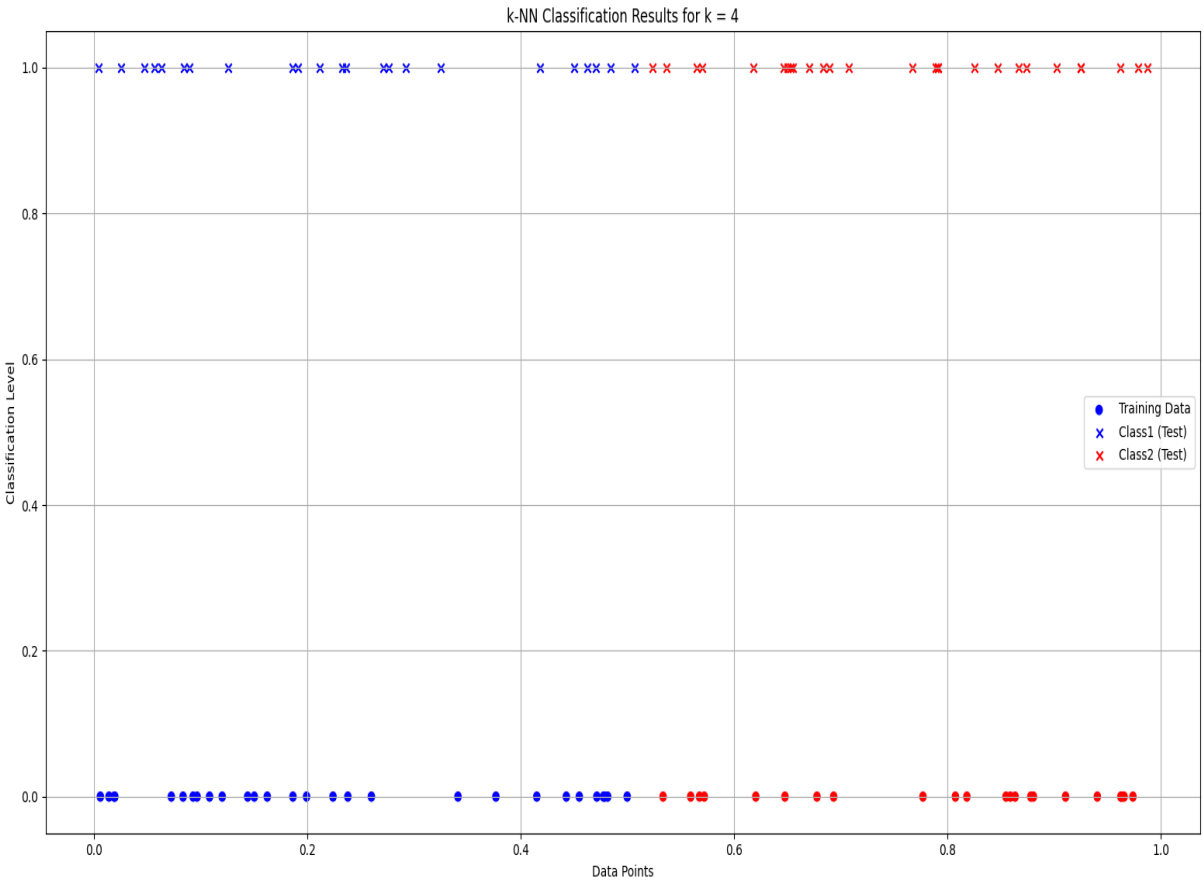
- **Low k(e.g., k=1,2)**:
  - Highly sensitive to local variations.
  - Prone to overfitting.
  - Can misclassify points near the decision boundary.
- **Moderate k (e.g., k=3,4,5)**:
  - Provides a balance between local sensitivity and generalization.
  - Reduces noise and improves stability.
- **High k (e.g., k=20)**:
  - Results in a **very smooth classification boundary**.
  - Can underfit the data as it prioritizes global trends over local patterns.

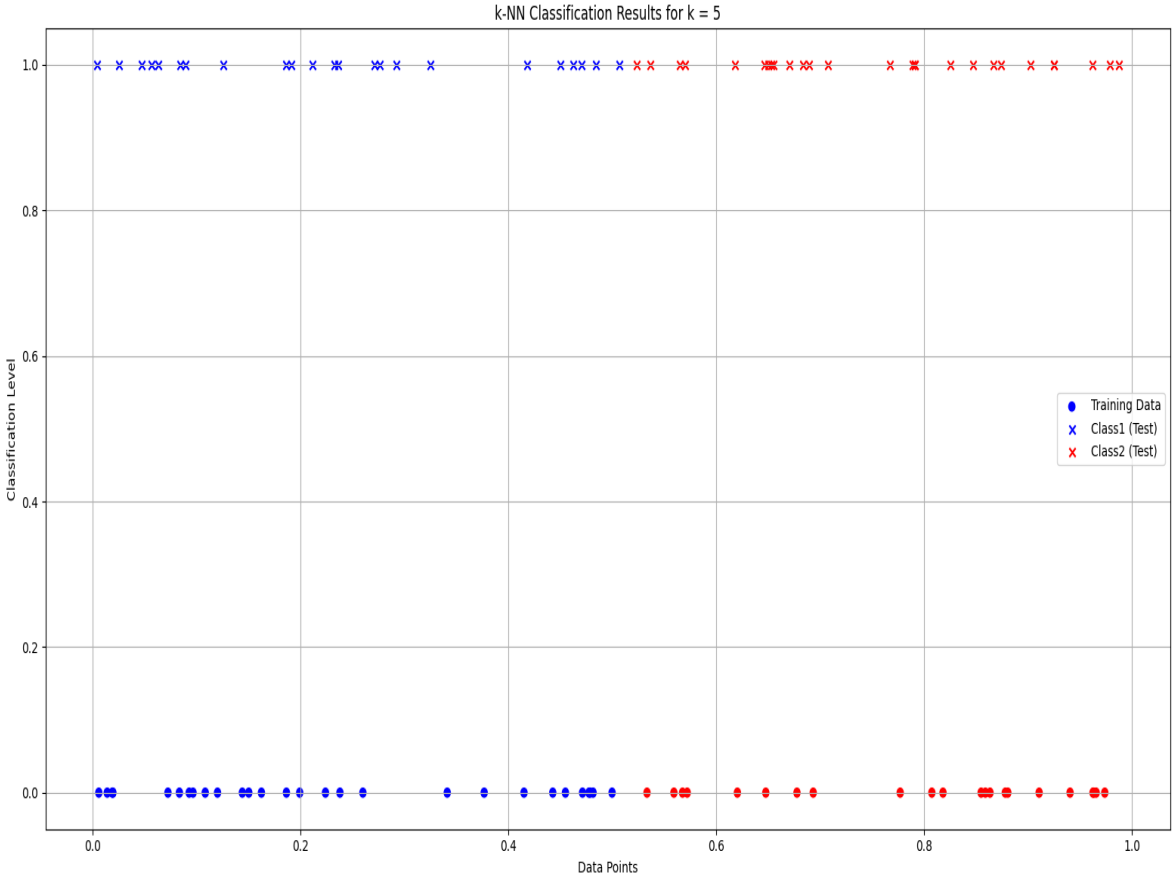
If the dataset has a **clear boundary**, choosing a small kmight work well. If the dataset has **some noise**, a moderate k (like 3-5) is ideal. If the dataset is **highly variable**, a larger k smooths the results but may **oversimplify** the classification.

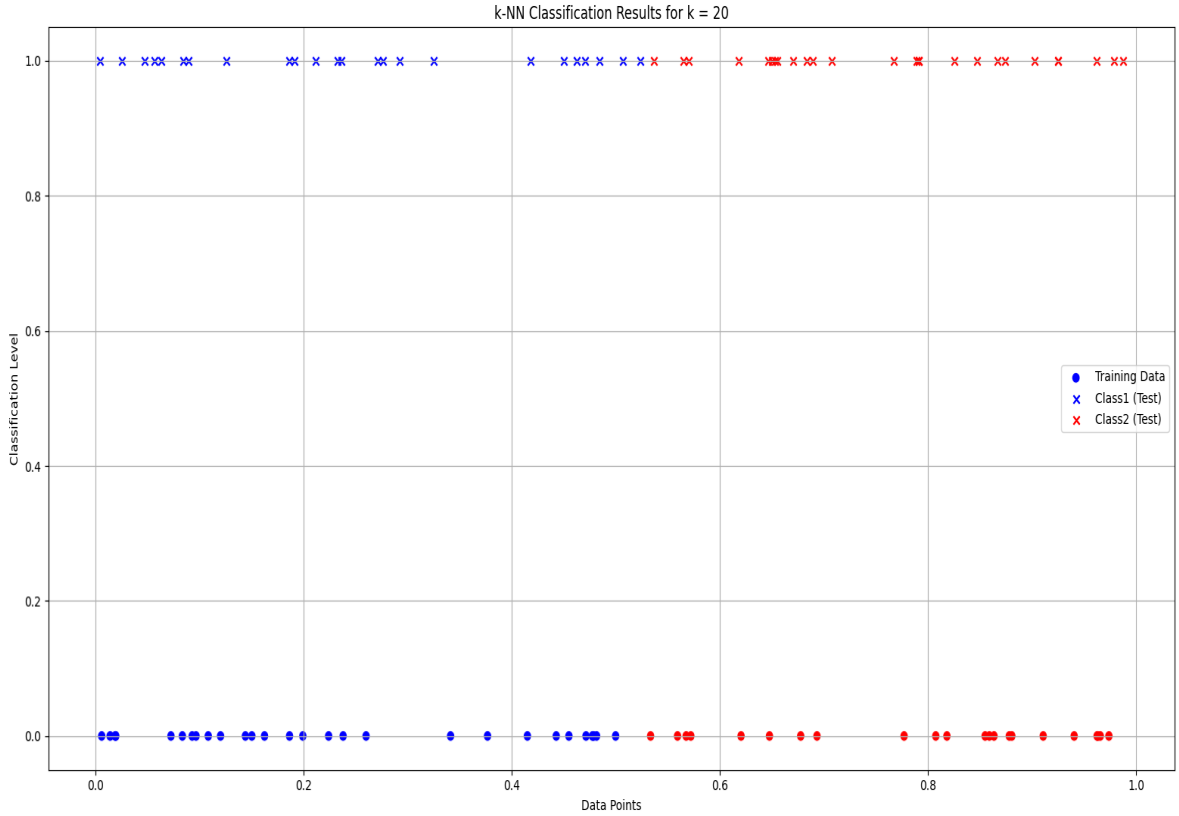


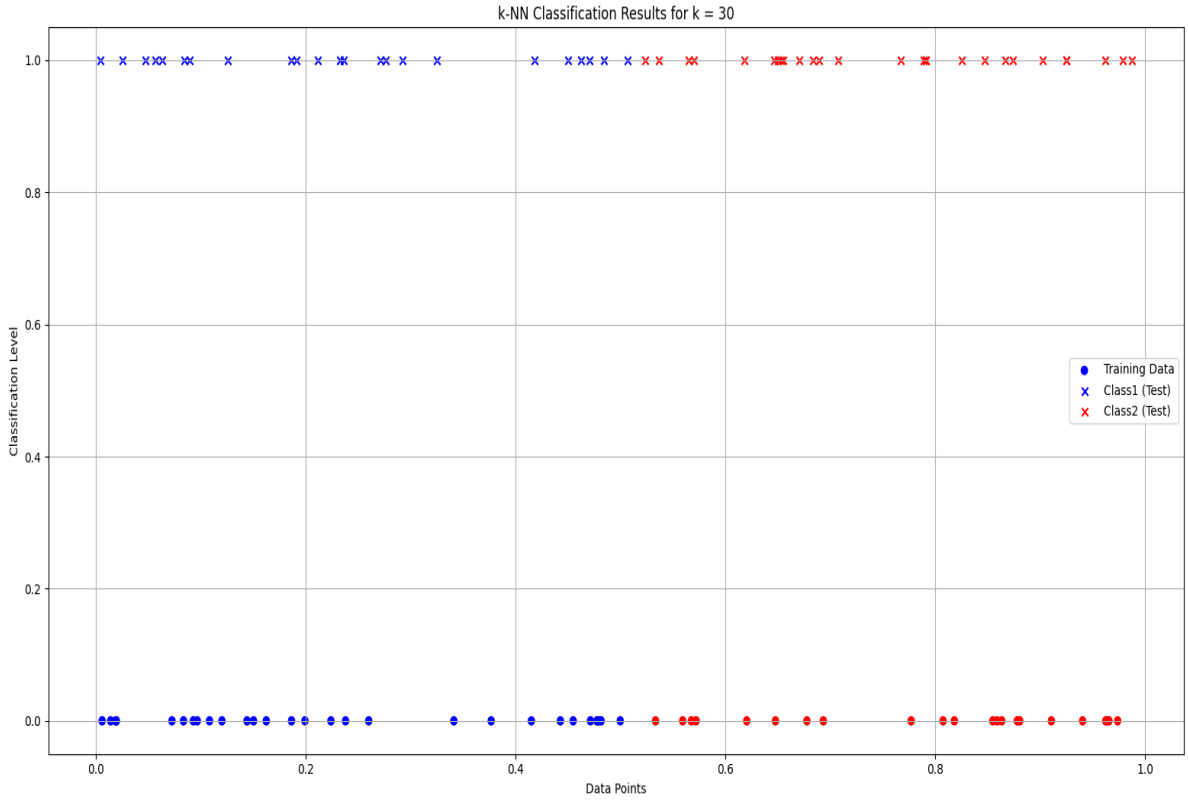

k-NN Classification Results for k = 2
Classification Level
Data Points
Training Data
Class1 (Test)
Class2 (Test)
0.0
0.2
0.4
0.6
0.8
1.0


k-NN Classification Results for k = 3
Classification Level
Training Data
Class1 (Test)
Class2 (Test)
0.0
0.2
0.4
0.6
0.8
1.0


k-NN Classification Results for k = 4
Classification Level
Data Points
Training Data
Class1 (Test)
Class2 (Test)
0.0
0.2
0.4
0.6
0.8
1.0


k-NN Classification Results for k = 5
Classification Level
Data Points
Training Data
Class1 (Test)
Class2 (Test)
0.0
0.2
0.4
0.6
0.8
1.0


k-NN Classification Results for k = 20
Classification Level
Data Points
Training Data
Class1 (Test)
Class2 (Test)
0.0
0.2
0.4
0.6
0.8
1.0


k-NN Classification Results for k = 30
Classification Level
Data Points
Training Data
Class1 (Test)
Class2 (Test)
0.0
0.2
0.4
0.6
0.8
1.0

```
Results for k = 1:
Point x51 (value: 0.6752) is classified as Class2
Point x52 (value: 0.9429) is classified as Class2
Point x53 (value: 0.3300) is classified as Class1
Point x54 (value: 0.7906) is classified as Class2
Point x55 (value: 0.8663) is classified as Class2
Point x56 (value: 0.0998) is classified as Class1
Point x57 (value: 0.6012) is classified as Class2
Point x58 (value: 0.4848) is classified as Class1
Point x59 (value: 0.9171) is classified as Class2
Point x60 (value: 0.2245) is classified as Class1
Point x61 (value: 0.0204) is classified as Class1
Point x62 (value: 0.1617) is classified as Class1
Point x63 (value: 0.4539) is classified as Class1
Point x64 (value: 0.1068) is classified as Class1
Point x65 (value: 0.1180) is classified as Class1
Point x66 (value: 0.2984) is classified as Class1
Point x67 (value: 0.6821) is classified as Class2
Point x68 (value: 0.7792) is classified as Class2
Point x69 (value: 0.5962) is classified as Class2
Point x70 (value: 0.5791) is classified as Class2
Point x71 (value: 0.3319) is classified as Class1
Point x72 (value: 0.1534) is classified as Class1
Point x73 (value: 0.8756) is classified as Class2
Point x74 (value: 0.7011) is classified as Class2
Point x75 (value: 0.5907) is classified as Class2
Point x76 (value: 0.2232) is classified as Class1
Point x77 (value: 0.6796) is classified as Class2
Point x78 (value: 0.4081) is classified as Class1
Point x79 (value: 0.7891) is classified as Class2
Point x80 (value: 0.0823) is classified as Class1
Point x81 (value: 0.8168) is classified as Class2
Point x82 (value: 0.8584) is classified as Class2
Point x83 (value: 0.1280) is classified as Class1
Point x84 (value: 0.7277) is classified as Class2
Point x85 (value: 0.6252) is classified as Class2
Point x86 (value: 0.3975) is classified as Class1
Point x87 (value: 0.5504) is classified as Class2
Point x88 (value: 0.6158) is classified as Class2
Point x89 (value: 0.6437) is classified as Class2
Point x90 (value: 0.0963) is classified as Class1
Point x91 (value: 0.6950) is classified as Class2
Point x92 (value: 0.8713) is classified as Class2
Point x93 (value: 0.3070) is classified as Class1
Point x94 (value: 0.2181) is classified as Class1
Point x95 (value: 0.7448) is classified as Class2
Point x96 (value: 0.8851) is classified as Class2
Point x97 (value: 0.3371) is classified as Class1
Point x98 (value: 0.6614) is classified as Class2
Point x99 (value: 0.7895) is classified as Class2
Point x100 (value: 0.3165) is classified as Class1
```

```
Results for k = 2:
Point x51 (value: 0.6752) is classified as Class2
Point x52 (value: 0.9429) is classified as Class2
Point x53 (value: 0.3300) is classified as Class1
Point x54 (value: 0.7906) is classified as Class2
Point x55 (value: 0.8663) is classified as Class2
Point x56 (value: 0.0998) is classified as Class1

Point x57 (value: 0.6012) is classified as Class2
Point x58 (value: 0.4848) is classified as Class1
Point x59 (value: 0.9171) is classified as Class2
Point x60 (value: 0.2245) is classified as Class1
Point x61 (value: 0.0204) is classified as Class1
Point x62 (value: 0.1617) is classified as Class1
Point x63 (value: 0.4539) is classified as Class1
Point x64 (value: 0.1068) is classified as Class1
Point x65 (value: 0.1180) is classified as Class1
Point x66 (value: 0.2984) is classified as Class1
Point x67 (value: 0.6821) is classified as Class2
Point x68 (value: 0.7792) is classified as Class2
Point x69 (value: 0.5962) is classified as Class2
Point x70 (value: 0.5791) is classified as Class2
Point x71 (value: 0.3319) is classified as Class1
Point x72 (value: 0.1534) is classified as Class1
Point x73 (value: 0.8756) is classified as Class2
Point x74 (value: 0.7011) is classified as Class2
Point x75 (value: 0.5907) is classified as Class2
Point x76 (value: 0.2232) is classified as Class1
Point x77 (value: 0.6796) is classified as Class2
Point x78 (value: 0.4081) is classified as Class1
Point x79 (value: 0.7891) is classified as Class2
Point x80 (value: 0.0823) is classified as Class1
Point x81 (value: 0.8168) is classified as Class2
Point x82 (value: 0.8584) is classified as Class2
Point x83 (value: 0.1280) is classified as Class1
Point x84 (value: 0.7277) is classified as Class2
Point x85 (value: 0.6252) is classified as Class2
Point x86 (value: 0.3975) is classified as Class1
Point x87 (value: 0.5504) is classified as Class2
Point x88 (value: 0.6158) is classified as Class2
Point x89 (value: 0.6437) is classified as Class2
Point x90 (value: 0.0963) is classified as Class1
Point x91 (value: 0.6950) is classified as Class2
Point x92 (value: 0.8713) is classified as Class2
Point x93 (value: 0.3070) is classified as Class1
Point x94 (value: 0.2181) is classified as Class1
Point x95 (value: 0.7448) is classified as Class2
Point x96 (value: 0.8851) is classified as Class2
Point x97 (value: 0.3371) is classified as Class1
Point x98 (value: 0.6614) is classified as Class2
Point x99 (value: 0.7895) is classified as Class2
Point x100 (value: 0.3165) is classified as Class1
```

```
Results for k = 3:
Point x51 (value: 0.6752) is classified as Class2
Point x52 (value: 0.9429) is classified as Class2
Point x53 (value: 0.3300) is classified as Class1
Point x54 (value: 0.7906) is classified as Class2
Point x55 (value: 0.8663) is classified as Class2
Point x56 (value: 0.0998) is classified as Class1
Point x57 (value: 0.6012) is classified as Class2
Point x58 (value: 0.4848) is classified as Class2
Point x59 (value: 0.9171) is classified as Class2
Point x60 (value: 0.2245) is classified as Class1
Point x61 (value: 0.0204) is classified as Class1
Point x62 (value: 0.1617) is classified as Class1
Point x63 (value: 0.4539) is classified as Class1
Point x64 (value: 0.1068) is classified as Class1
Point x65 (value: 0.1180) is classified as Class1
Point x66 (value: 0.2984) is classified as Class1
Point x67 (value: 0.6821) is classified as Class2
Point x68 (value: 0.7792) is classified as Class2
Point x69 (value: 0.5962) is classified as Class2
Point x70 (value: 0.5791) is classified as Class2

Point x71 (value: 0.3319) is classified as Class1
Point x72 (value: 0.1534) is classified as Class1
Point x73 (value: 0.8756) is classified as Class2
Point x74 (value: 0.7011) is classified as Class2
Point x75 (value: 0.5907) is classified as Class2
Point x76 (value: 0.2232) is classified as Class1
Point x77 (value: 0.6796) is classified as Class2
Point x78 (value: 0.4081) is classified as Class1
Point x79 (value: 0.7891) is classified as Class2
Point x80 (value: 0.0823) is classified as Class1
Point x81 (value: 0.8168) is classified as Class2
Point x82 (value: 0.8584) is classified as Class2
Point x83 (value: 0.1280) is classified as Class1
Point x84 (value: 0.7277) is classified as Class2
Point x85 (value: 0.6252) is classified as Class2
Point x86 (value: 0.3975) is classified as Class1
Point x87 (value: 0.5504) is classified as Class2
Point x88 (value: 0.6158) is classified as Class2
Point x89 (value: 0.6437) is classified as Class2
Point x90 (value: 0.0963) is classified as Class1
Point x91 (value: 0.6950) is classified as Class2
Point x92 (value: 0.8713) is classified as Class2
Point x93 (value: 0.3070) is classified as Class1
Point x94 (value: 0.2181) is classified as Class1
Point x95 (value: 0.7448) is classified as Class2
Point x96 (value: 0.8851) is classified as Class2
Point x97 (value: 0.3371) is classified as Class1
Point x98 (value: 0.6614) is classified as Class2
Point x99 (value: 0.7895) is classified as Class2
Point x100 (value: 0.3165) is classified as Class1
```

```
Results for k = 4:
Point x51 (value: 0.6752) is classified as Class2
Point x52 (value: 0.9429) is classified as Class2
Point x53 (value: 0.3300) is classified as Class1
Point x54 (value: 0.7906) is classified as Class2
Point x55 (value: 0.8663) is classified as Class2
Point x56 (value: 0.0998) is classified as Class1
Point x57 (value: 0.6012) is classified as Class2
Point x58 (value: 0.4848) is classified as Class1
Point x59 (value: 0.9171) is classified as Class2
Point x60 (value: 0.2245) is classified as Class1
Point x61 (value: 0.0204) is classified as Class1
Point x62 (value: 0.1617) is classified as Class1
Point x63 (value: 0.4539) is classified as Class1
Point x64 (value: 0.1068) is classified as Class1
Point x65 (value: 0.1180) is classified as Class1
Point x66 (value: 0.2984) is classified as Class1
Point x67 (value: 0.6821) is classified as Class2
Point x68 (value: 0.7792) is classified as Class2
Point x69 (value: 0.5962) is classified as Class2
Point x70 (value: 0.5791) is classified as Class2
Point x71 (value: 0.3319) is classified as Class1
Point x72 (value: 0.1534) is classified as Class1
Point x73 (value: 0.8756) is classified as Class2
Point x74 (value: 0.7011) is classified as Class2
Point x75 (value: 0.5907) is classified as Class2
Point x76 (value: 0.2232) is classified as Class1
Point x77 (value: 0.6796) is classified as Class2
Point x78 (value: 0.4081) is classified as Class1
Point x79 (value: 0.7891) is classified as Class2
Point x80 (value: 0.0823) is classified as Class1
Point x81 (value: 0.8168) is classified as Class2
Point x82 (value: 0.8584) is classified as Class2
Point x83 (value: 0.1280) is classified as Class1
Point x84 (value: 0.7277) is classified as Class2

Point x85 (value: 0.6252) is classified as Class2
Point x86 (value: 0.3975) is classified as Class1
Point x87 (value: 0.5504) is classified as Class2
Point x88 (value: 0.6158) is classified as Class2
Point x89 (value: 0.6437) is classified as Class2
Point x90 (value: 0.0963) is classified as Class1
Point x91 (value: 0.6950) is classified as Class2
Point x92 (value: 0.8713) is classified as Class2
Point x93 (value: 0.3070) is classified as Class1
Point x94 (value: 0.2181) is classified as Class1
Point x95 (value: 0.7448) is classified as Class2
Point x96 (value: 0.8851) is classified as Class2
Point x97 (value: 0.3371) is classified as Class1
Point x98 (value: 0.6614) is classified as Class2
Point x99 (value: 0.7895) is classified as Class2
Point x100 (value: 0.3165) is classified as Class1
```

```
Results for k = 5:
Point x51 (value: 0.6752) is classified as Class2
Point x52 (value: 0.9429) is classified as Class2
Point x53 (value: 0.3300) is classified as Class1
Point x54 (value: 0.7906) is classified as Class2
Point x55 (value: 0.8663) is classified as Class2
Point x56 (value: 0.0998) is classified as Class1
Point x57 (value: 0.6012) is classified as Class2
Point x58 (value: 0.4848) is classified as Class2
Point x59 (value: 0.9171) is classified as Class2
Point x60 (value: 0.2245) is classified as Class1
Point x61 (value: 0.0204) is classified as Class1
Point x62 (value: 0.1617) is classified as Class1
Point x63 (value: 0.4539) is classified as Class1
Point x64 (value: 0.1068) is classified as Class1
Point x65 (value: 0.1180) is classified as Class1
Point x66 (value: 0.2984) is classified as Class1
Point x67 (value: 0.6821) is classified as Class2
Point x68 (value: 0.7792) is classified as Class2
Point x69 (value: 0.5962) is classified as Class2
Point x70 (value: 0.5791) is classified as Class2
Point x71 (value: 0.3319) is classified as Class1
Point x72 (value: 0.1534) is classified as Class1
Point x73 (value: 0.8756) is classified as Class2
Point x74 (value: 0.7011) is classified as Class2
Point x75 (value: 0.5907) is classified as Class2
Point x76 (value: 0.2232) is classified as Class1
Point x77 (value: 0.6796) is classified as Class2
Point x78 (value: 0.4081) is classified as Class1
Point x79 (value: 0.7891) is classified as Class2
Point x80 (value: 0.0823) is classified as Class1
Point x81 (value: 0.8168) is classified as Class2
Point x82 (value: 0.8584) is classified as Class2
Point x83 (value: 0.1280) is classified as Class1
Point x84 (value: 0.7277) is classified as Class2
Point x85 (value: 0.6252) is classified as Class2
Point x86 (value: 0.3975) is classified as Class1
Point x87 (value: 0.5504) is classified as Class2
Point x88 (value: 0.6158) is classified as Class2
Point x89 (value: 0.6437) is classified as Class2
Point x90 (value: 0.0963) is classified as Class1
Point x91 (value: 0.6950) is classified as Class2
Point x92 (value: 0.8713) is classified as Class2
Point x93 (value: 0.3070) is classified as Class1
Point x94 (value: 0.2181) is classified as Class1
Point x95 (value: 0.7448) is classified as Class2
Point x96 (value: 0.8851) is classified as Class2
Point x97 (value: 0.3371) is classified as Class1
Point x98 (value: 0.6614) is classified as Class2

Point x99 (value: 0.7895) is classified as Class2
Point x100 (value: 0.3165) is classified as Class1
```

```
Results for k = 20:
Point x51 (value: 0.6752) is classified as Class2
Point x52 (value: 0.9429) is classified as Class2
Point x53 (value: 0.3300) is classified as Class1
Point x54 (value: 0.7906) is classified as Class2
Point x55 (value: 0.8663) is classified as Class2
Point x56 (value: 0.0998) is classified as Class1
Point x57 (value: 0.6012) is classified as Class2
Point x58 (value: 0.4848) is classified as Class1
Point x59 (value: 0.9171) is classified as Class2
Point x60 (value: 0.2245) is classified as Class1
Point x61 (value: 0.0204) is classified as Class1
Point x62 (value: 0.1617) is classified as Class1
Point x63 (value: 0.4539) is classified as Class1
Point x64 (value: 0.1068) is classified as Class1
Point x65 (value: 0.1180) is classified as Class1
Point x66 (value: 0.2984) is classified as Class1
Point x67 (value: 0.6821) is classified as Class2
Point x68 (value: 0.7792) is classified as Class2
Point x69 (value: 0.5962) is classified as Class2
Point x70 (value: 0.5791) is classified as Class2
Point x71 (value: 0.3319) is classified as Class1
Point x72 (value: 0.1534) is classified as Class1
Point x73 (value: 0.8756) is classified as Class2
Point x74 (value: 0.7011) is classified as Class2
Point x75 (value: 0.5907) is classified as Class2
Point x76 (value: 0.2232) is classified as Class1
Point x77 (value: 0.6796) is classified as Class2
Point x78 (value: 0.4081) is classified as Class1
Point x79 (value: 0.7891) is classified as Class2
Point x80 (value: 0.0823) is classified as Class1
Point x81 (value: 0.8168) is classified as Class2
Point x82 (value: 0.8584) is classified as Class2
Point x83 (value: 0.1280) is classified as Class1
Point x84 (value: 0.7277) is classified as Class2
Point x85 (value: 0.6252) is classified as Class2
Point x86 (value: 0.3975) is classified as Class1
Point x87 (value: 0.5504) is classified as Class2
Point x88 (value: 0.6158) is classified as Class2
Point x89 (value: 0.6437) is classified as Class2
Point x90 (value: 0.0963) is classified as Class1
Point x91 (value: 0.6950) is classified as Class2
Point x92 (value: 0.8713) is classified as Class2
Point x93 (value: 0.3070) is classified as Class1
Point x94 (value: 0.2181) is classified as Class1
Point x95 (value: 0.7448) is classified as Class2
Point x96 (value: 0.8851) is classified as Class2
Point x97 (value: 0.3371) is classified as Class1
Point x98 (value: 0.6614) is classified as Class2
Point x99 (value: 0.7895) is classified as Class2
Point x100 (value: 0.3165) is classified as Class1
```

```
Results for k = 30:
Point x51 (value: 0.6752) is classified as Class2
Point x52 (value: 0.9429) is classified as Class2
Point x53 (value: 0.3300) is classified as Class1
Point x54 (value: 0.7906) is classified as Class2
Point x55 (value: 0.8663) is classified as Class2
Point x56 (value: 0.0998) is classified as Class1
Point x57 (value: 0.6012) is classified as Class2
Point x58 (value: 0.4848) is classified as Class1
Point x59 (value: 0.9171) is classified as Class2

Point x60 (value: 0.2245) is classified as Class1
Point x61 (value: 0.0204) is classified as Class1
Point x62 (value: 0.1617) is classified as Class1
Point x63 (value: 0.4539) is classified as Class1
Point x64 (value: 0.1068) is classified as Class1
Point x65 (value: 0.1180) is classified as Class1
Point x66 (value: 0.2984) is classified as Class1
Point x67 (value: 0.6821) is classified as Class2
Point x68 (value: 0.7792) is classified as Class2
Point x69 (value: 0.5962) is classified as Class2
Point x70 (value: 0.5791) is classified as Class2
Point x71 (value: 0.3319) is classified as Class1
Point x72 (value: 0.1534) is classified as Class1
Point x73 (value: 0.8756) is classified as Class2
Point x74 (value: 0.7011) is classified as Class2
Point x75 (value: 0.5907) is classified as Class2
Point x76 (value: 0.2232) is classified as Class1
Point x77 (value: 0.6796) is classified as Class2
Point x78 (value: 0.4081) is classified as Class1
Point x79 (value: 0.7891) is classified as Class2
Point x80 (value: 0.0823) is classified as Class1
Point x81 (value: 0.8168) is classified as Class2
Point x82 (value: 0.8584) is classified as Class2
Point x83 (value: 0.1280) is classified as Class1
Point x84 (value: 0.7277) is classified as Class2
Point x85 (value: 0.6252) is classified as Class2
Point x86 (value: 0.3975) is classified as Class1
Point x87 (value: 0.5504) is classified as Class2
Point x88 (value: 0.6158) is classified as Class2
Point x89 (value: 0.6437) is classified as Class2
Point x90 (value: 0.0963) is classified as Class1
Point x91 (value: 0.6950) is classified as Class2
Point x92 (value: 0.8713) is classified as Class2
Point x93 (value: 0.3070) is classified as Class1
Point x94 (value: 0.2181) is classified as Class1
Point x95 (value: 0.7448) is classified as Class2
Point x96 (value: 0.8851) is classified as Class2
Point x97 (value: 0.3371) is classified as Class1
Point x98 (value: 0.6614) is classified as Class2
Point x99 (value: 0.7895) is classified as Class2
Point x100 (value: 0.3165) is classified as Class1
```

6.Implement the non-parametric Locally Weighted Regression algorithm in order to fit data points. Select appropriate data set for your experiment and draw graphs.

AIM:

To implement Locally Weighted Regression (LWR) using a Gaussian kernel, fit the model to a noisy sine wave dataset, and visualize the resulting fit. The program demonstrates how LWR can be used to create smooth approximations of noisy data by weighing training points based on their proximity to the query point.

OBJECTIVES:

- **Generate Data:**
  Create a noisy sine wave dataset to be used for training and testing the LWR model.
- **Define Gaussian Kernel Function:**
  Implement a function to calculate the Gaussian kernel, which will be used to weigh training points.
- **Implement LWR Function:**
  Develop the Locally Weighted Regression function to fit a model to the data, considering weights based on the Gaussian kernel.
- **Make Predictions:**
  Use the LWR function to make predictions for new data points, creating a smooth curve that approximates the underlying sine wave.
- **Visualize Results:** Plot the training data and the LWR fit to visually compare the noisy data and the smooth approximation.

PROGRAM:

```
import numpy as np

import matplotlib.pyplot as plt

# Define Gaussian kernel function

def gaussian_kernel(x, xi, tau):

# Calculate the Gaussian kernel (similarity) between point x and xi with bandwidth tau
```

```
    return np.exp(-np.sum((x - xi) ** 2) / (2 * tau ** 2))

# Define Locally Weighted Regression (LWR) function

def locally_weighted_regression(x, X, y, tau):

    m = X.shape[0]

    # Calculate weights using the Gaussian kernel for all training points with respect to x

    weights = np.array([gaussian_kernel(x, X[i], tau) for i in range(m)])

    # Create a diagonal matrix W with the calculated weights

    W = np.diag(weights)

    # Calculate the weighted design matrix X^T * W

    X_transpose_W = X.T @ W

    # Compute the coefficients theta by solving the weighted least squares problem

    theta = np.linalg.inv(X_transpose_W @ X) @ X_transpose_W @ y

    # Return the prediction for x

    return x @ theta

# Set random seed for reproducibility

np.random.seed(42)

# Generate 100 evenly spaced points between 0 and 2π

X = np.linspace(0, 2 * np.pi, 100)

# Generate noisy sine wave data

y = np.sin(X) + 0.1 * np.random.randn(100)

# Add a bias term (column of ones) to the training data

X_bias = np.c_[np.ones(X.shape), X]
```

```
# Generate 200 test points between 0 and 2π

x_test = np.linspace(0, 2 * np.pi, 200)

# Add a bias term (column of ones) to the test data

x_test_bias = np.c_[np.ones(x_test.shape), x_test]

# Set the bandwidth parameter tau

tau = 0.5

# Make predictions for each test point using the LWR function

y_pred = np.array([locally_weighted_regression(xi, X_bias, y, tau) for xi in x_test_bias])

# Create a plot of the results

plt.figure(figsize=(10, 6))

# Plot the training data points as red scatter points

plt.scatter(X, y, color='red', label='Training Data', alpha=0.7)

# Plot the LWR fit as a blue line

plt.plot(x_test, y_pred, color='blue', label=f'LWR Fit (tau={tau})', linewidth=2)

# Add labels, title, legend, and grid to the plot

plt.xlabel('X', fontsize=12)

plt.ylabel('y', fontsize=12)

plt.title('Locally Weighted Regression', fontsize=14)

plt.legend(fontsize=10)

plt.grid(alpha=0.3)

# Display the plot
```

plt.show()

EXPLANATION:

Importing Libraries:

numpy: A library for numerical operations and handling arrays.

matplotlib.pyplot: A library for plotting graphs and visualizing data.

Defining the Gaussian Kernel Function:

```
def gaussian_kernel(x, xi, tau):  1 usage
    return np.exp(-np.sum((x - xi) ** 2) / (2 * tau ** 2))
```

This function calculates the Gaussian kernel, which is a measure of similarity between two points x and xi.

The parameter tau controls the bandwidth of the kernel. Smaller tau results in a narrower kernel, giving more weight to closer points.

Defining the Locally Weighted Regression (LWR) Function:

```
def locally_weighted_regression(x, X, y, tau):  1 usage
    m = X.shape[0]
    weights = np.array([gaussian_kernel(x, X[i], tau) for i in range(m)])
    W = np.diag(weights)
    X_transpose_W = X.T @ W
    theta = np.linalg.inv(X_transpose_W @ X) @ X_transpose_W @ y
    return x @ theta
```

x: The query point at which we want to make a prediction.

X: The training data points.

y: The target values for the training data.

tau: The bandwidth parameter.

The function calculates weights for each training point using the Gaussian kernel.

W: A diagonal matrix with the calculated weights.

X_transpose_W: The product of X transposed and W.

theta: The coefficients obtained by solving the weighted least squares problem using matrix operations.

The function returns the prediction for x.

Generating Data:

```
np.random.seed(42)
X = np.linspace( start: 0, 2 * np.pi,  num: 100)
y = np.sin(X) + 0.1 * np.random.randn(100)
```

np.random.seed(42): Sets the random seed for reproducibility, ensuring the same random values are generated each time.

X: Generates 100 evenly spaced points between 0 and 2π2\pi.

y: Generates noisy sine wave data by adding random noise to the sine of X.

<u>Adding Bias Term to Training Data</u>:

```
X_bias = np.c_[np.ones(X.shape), X]
```

X_bias: Adds a column of ones to X to account for the bias term in linear regression.

<u>Generating Test Data and Adding Bias Term:</u>

```
x_test = np.linspace( start: 0, 2 * np.pi,  num: 200)
x_test_bias = np.c_[np.ones(x_test.shape), x_test]
```

x_test: Generates 200 evenly spaced points between 0 and 2π2\pi.

x_test_bias: Adds a column of ones to x_test to account for the bias term in linear regression.

<u>Setting Bandwidth Parameter and Making Predictions</u>:

```
tau = 0.5
y_pred = np.array([locally_weighted_regression(xi, X_bias, y, tau) for xi in x_test_bias])
```

tau = 0.5: Sets the bandwidth parameter, controlling the influence of nearby points.

y_pred: Calculates predictions for each point in x_test_bias using the LWR function, generating a smooth approximation of the noisy sine wave data.

<u>Plotting the Results</u>:

```
plt.figure(figsize=(10, 6))
plt.scatter(X, y, color='red', label='Training Data', alpha=0.7)
plt.plot( *args: x_test, y_pred, color='blue', label=f'LWR Fit (tau={tau})', linewidth=2)
plt.xlabel( xlabel: 'X', fontsize=12)
plt.ylabel( ylabel: 'y', fontsize=12)
plt.title( label: 'Locally Weighted Regression', fontsize=14)
plt.legend(fontsize=10)
plt.grid(alpha=0.3)
plt.show()
```

plt.figure(figsize=(10, 6)): Creates a figure with a specified size.

plt.scatter(X, y, color='red', label='Training Data', alpha=0.7): Plots the training data points as red scatter points with some transparency (alpha).

plt.plot(x_test, y_pred, color='blue', label=f'LWR Fit (tau={tau})', linewidth=2): Plots the LWR fit as a blue line.

plt.xlabel('X', fontsize=12): Adds an X-axis label.

plt.ylabel('y', fontsize=12): Adds a Y-axis label.

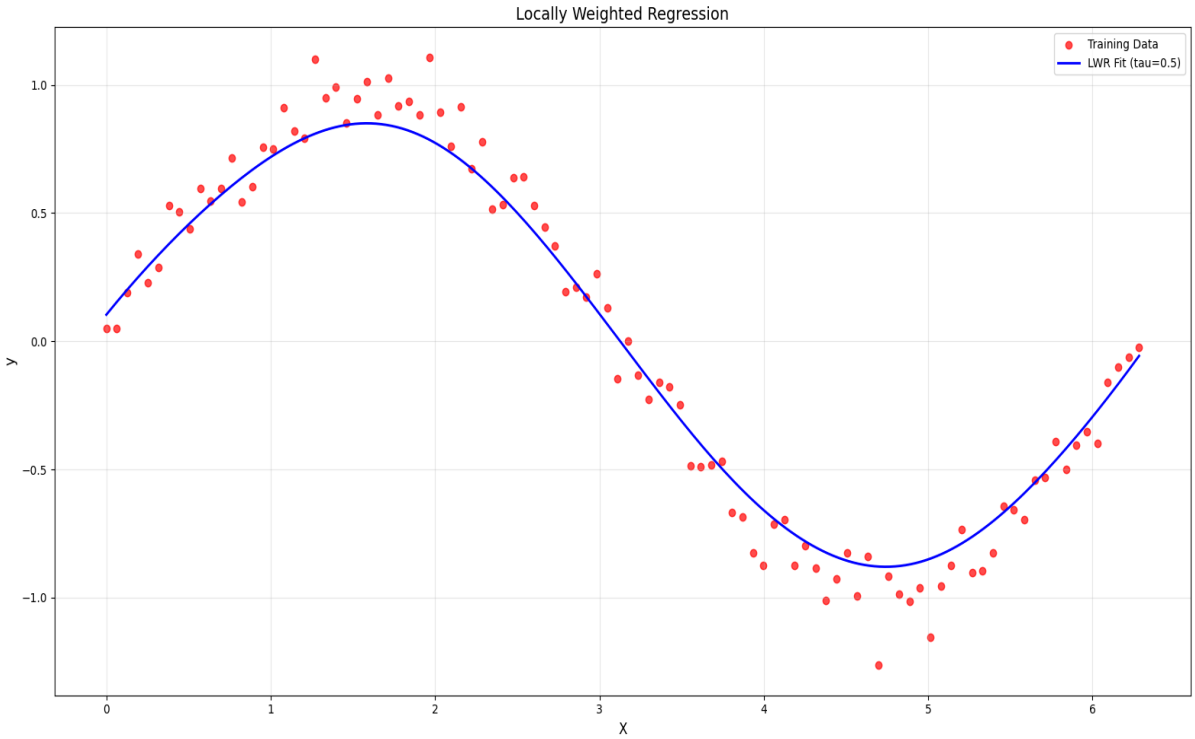
plt.title('Locally Weighted Regression', fontsize=14): Adds a title to the plot.

plt.legend(fontsize=10): Adds a legend with specified font size.

plt.grid(alpha=0.3): Adds a grid with some transparency**.**

plt.show(): Displays the plot.

OUTPUT:



The output of the program is a plot that visualizes the results of Locally Weighted Regression (LWR) applied to a noisy sine wave dataset. Let's analyze and explain the various components of the plot:

1. **Training Data:**

- **Red Scatter Points:** These points represent the noisy sine wave data used to train the LWR model. The X-axis values range from 0 to 2π2\pi, and the Y-axis values are the sine of the X values with added Gaussian noise. The noise simulates real-world data that often contains some randomness or variability.

2. **LWR Fit:**

   - **Blue Curve:** This smooth curve represents the fit of the LWR model to the noisy training data. The LWR model makes predictions for new data points by considering the weights of nearby training points. The bandwidth parameter (tau) controls the influence of nearby points. In this case, tau is set to 0.5, resulting in a fit that balances smoothness and adherence to the training data.

3. **X-Axis and Y-Axis:**

   - **X-Axis (X):** Represents the input values, which range from 0 to 2π2\pi. These are the points at which the sine function is evaluated.
   - **Y-Axis (y):** Represents the output values of the sine function with added noise. These values are used as the target values for the LWR model.

4. **Plot Details:**

   - **Title:** "Locally Weighted Regression" – indicates the type of regression applied to the data.
   - **Labels:** The X-axis is labeled "X" and the Y-axis is labeled "y". These labels provide context for the input and output values.
   - **Legend:** The legend differentiates between the training data (red scatter points) and the LWR fit (blue curve). The legend also includes the value of tau used in the LWR model.
   - **Grid:** A light grid is added to the plot for better visualization of data points and the fit.

Explanation of the Fit:

- The LWR Fit (blue curve) closely follows the general trend of the noisy sine wave data (red points).
- The smooth curve suggests that the LWR model has successfully captured the underlying sine wave pattern despite the presence of noise.
- The chosen tau value (0.5) results in a fit that is neither too rigid nor too flexible. A smaller tau would lead to a more flexible fit, potentially overfitting the noise, while a larger tau would lead to a smoother fit, potentially underfitting the data.

The plot effectively demonstrates the capability of Locally Weighted Regression to handle noisy data and provide a smooth approximation of the underlying pattern. By assigning higher weights to nearby points, the LWR model is able to make accurate local predictions while maintaining overall smoothness.

7.Develop a program to demonstrate the working of Linear Regression and Polynomial Regression. Use Boston Housing Dataset for Linear Regression and Auto MPG Dataset (for vehicle fuel efficiency prediction) for Polynomial Regression.

AIM:

To demonstrate the application of linear regression and polynomial regression techniques using two different datasets: the California Housing dataset and the Auto MPG dataset. The program aims to perform these regressions, visualize the actual vs. predicted values, and evaluate the model performance using appropriate metrics.

OBJECTIVES:

- **Demonstrate Regression Techniques:**
  Show the application of both linear and polynomial regression techniques on real-world datasets.
- **Model Evaluation:**
  Highlight the importance of evaluating regression models using both visual and quantitative metrics.
- **Visualization:**
  Provide clear visualizations of the regression results to aid in understanding and interpretation of the model's performance.
- **Practical Implementation:**
  Illustrate the steps involved in implementing linear and polynomial regression, from data loading and preprocessing to model training, prediction, visualization, and evaluation.

PROGRAM:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
```

```
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.metrics import mean_squared_error, r2_score

# Function to perform linear regression on the California Housing dataset
def linear_regression_california():
    # Load the California Housing dataset
    housing = fetch_california_housing(as_frame=True)
    X = housing.data[["AveRooms"]]  # Select the feature 'Average number of rooms'
    y = housing.target  # Target variable (median house value)

    # Split the data into training and testing sets
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    # Create and train the linear regression model
    model = LinearRegression()
    model.fit(X_train, y_train)

    # Predict the target variable for the test set
    y_pred = model.predict(X_test)

    # Plot the actual vs. predicted values
    plt.scatter(X_test, y_test, color="blue", label="Actual")
    plt.plot(X_test, y_pred, color="red", label="Predicted")
    plt.xlabel("Average number of rooms (AveRooms)")
    plt.ylabel("Median value of homes ($100,000)")
    plt.title("Linear Regression - California Housing Dataset")
    plt.legend()
    plt.show()

    # Print the performance metrics
    print("Linear Regression - California Housing Dataset")
```

```
    print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
    print("R^2 Score:", r2_score(y_test, y_pred))

# Function to perform polynomial regression on the Auto MPG dataset
def polynomial_regression_auto_mpg():
    # URL for the Auto MPG dataset
    url = https://archive.ics.uci.edu/ml/machine-learning-databases/auto-mpg/auto-mpg.data

    # Column names for the dataset
    column_names = ["mpg", "cylinders", "displacement", "horsepower", "weight",
"acceleration", "model_year", "origin"]

    # Load the dataset
    data = pd.read_csv(url, sep=r'\s+', names=column_names, na_values="?")
    data = data.dropna()  # Drop rows with missing values

    # Select the feature 'displacement' and target variable 'mpg'
    X = data["displacement"].values.reshape(-1, 1)
    y = data["mpg"].values

    # Split the data into training and testing sets
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    # Create and train the polynomial regression model (degree=2) with standard scaling
    poly_model = make_pipeline(PolynomialFeatures(degree=2), StandardScaler(),
LinearRegression())
    poly_model.fit(X_train, y_train)

    # Predict the target variable for the test set
    y_pred = poly_model.predict(X_test)

    # Plot the actual vs. predicted values
    plt.scatter(X_test, y_test, color="blue", label="Actual")
```

```
    plt.scatter(X_test, y_pred, color="red", label="Predicted")
    plt.xlabel("Displacement")
    plt.ylabel("Miles per gallon (mpg)")
    plt.title("Polynomial Regression - Auto MPG Dataset")
    plt.legend()
    plt.show()

    # Print the performance metrics
    print("Polynomial Regression - Auto MPG Dataset")
    print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
    print("R^2 Score:", r2_score(y_test, y_pred))

# Main function to demonstrate linear and polynomial regression
if __name__ == "__main__":
    print("Demonstrating Linear Regression and Polynomial Regression\n")
    linear_regression_california()
    polynomial_regression_auto_mpg()
```

EXPLANATION:

Importing Libraries:

- **numpy, pandas, matplotlib.pyplot:** Libraries for numerical operations, data manipulation, and plotting.
- **sklearn:** A machine learning library for data preprocessing, model training, and evaluation.

Linear Regression - California Housing Dataset:

- **fetch_california_housing:** Loads the California Housing dataset.
- **train_test_split:** Splits the dataset into training and testing sets.
- **LinearRegression:** Creates a linear regression model.
- **model.fit:** Trains the linear regression model on the training data.
- **model.predict:** Predicts the target variable for the test set.
- **plt.scatter, plt.plot:** Plots the actual vs. predicted values.

- **mean_squared_error, r2_score:** Evaluates the model's performance using MSE and R^2 score.

Polynomial Regression - Auto MPG Dataset:

- **pd.read_csv:** Loads the Auto MPG dataset.
- **data.dropna:** Drops rows with missing values.
- **PolynomialFeatures:** Generates polynomial features for the input data.
- **StandardScaler:** Standardizes the features by removing the mean and scaling to unit variance.
- **make_pipeline:** Creates a pipeline that combines polynomial feature generation, standard scaling, and linear regression.
- **poly_model.fit:** Trains the polynomial regression model on the training data.
- **poly_model.predict:** Predicts the target variable for the test set.
- plt.scatter: Plots the actual vs. predicted values.
- **mean_squared_error, r2_score:** Evaluates the model's performance using MSE and R^2 score.

Main Function:

- Demonstrates the linear regression on the California Housing dataset and polynomial regression on the Auto MPG dataset by calling the respective functions.

This program demonstrates how to perform linear and polynomial regression on different datasets and evaluates the model's performance using appropriate metrics.

OUTPUT:

Figure7.1 shows the output for the program. The output of the program demonstrates the effectiveness of linear and polynomial regression on different datasets. In the case of **Linear Regression using the California Housing dataset**, the model was trained using only one feature, **"AveRooms"** (average number of rooms per dwelling), to predict median house values. The **Mean Squared Error (MSE)** of **1.2923** indicates a relatively high error in predictions, and the **R^2 score of 0.0138** suggests that the model explains only **1.38%** of the variance in house prices. This poor performance is likely due to the fact that house prices are

influenced by multiple factors such as income levels, crime rates, and proximity to amenities, which were not considered in this model. On the other hand, the **Polynomial Regression on the Auto MPG dataset** shows significantly better results. The model used the **"Displacement"** feature to predict miles per gallon (MPG), achieving an **MSE of 0.7431** and an **R^2 score of 0.7506**, indicating that the model explains **75.06%** of the variance in fuel efficiency. This suggests that polynomial regression is a better fit for this dataset, likely due to the non-linear relationship between engine displacement and fuel efficiency. Overall, while **linear regression struggled due to an insufficient feature set**, polynomial regression demonstrated a strong correlation, emphasizing the importance of selecting appropriate features and models for different datasets. To improve the performance of linear regression on the California Housing dataset, it would be beneficial to include additional relevant features such as **income levels, population, and location-related attributes**.

```
Demonstrating Linear Regression and Polynomial Regression


Linear Regression - California Housing Dataset
Mean Squared Error: 1.2923314440807299
R^2 Score: 0.013795337532284901
Polynomial Regression - Auto MPG Dataset
Mean Squared Error: 0.743149055720586
R^2 Score: 0.7505650609469626
```

Figure7.1: output for the program

The plot in the Figure7.2 visualizes the Linear Regression model applied to the California Housing dataset, where the x-axis represents the average number of rooms per dwelling (AveRooms) and the y-axis represents the median house value (in $100,000s). The blue scatter points depict the actual house prices, while the red line represents the model's predicted values.

From the visualization, it is evident that the model does not perform well. The predicted trend line (red) does not closely fit the actual data points, suggesting a poor linear relationship between the number of rooms and house prices. Most data points are clustered around lower

values of "AveRooms" (0-10), but the model extends the prediction line unrealistically for higher room counts, leading to inaccurate predictions. This aligns with the low R^2 score of 0.0138 observed earlier, indicating that only 1.38% of the variance in house prices is explained by the model.

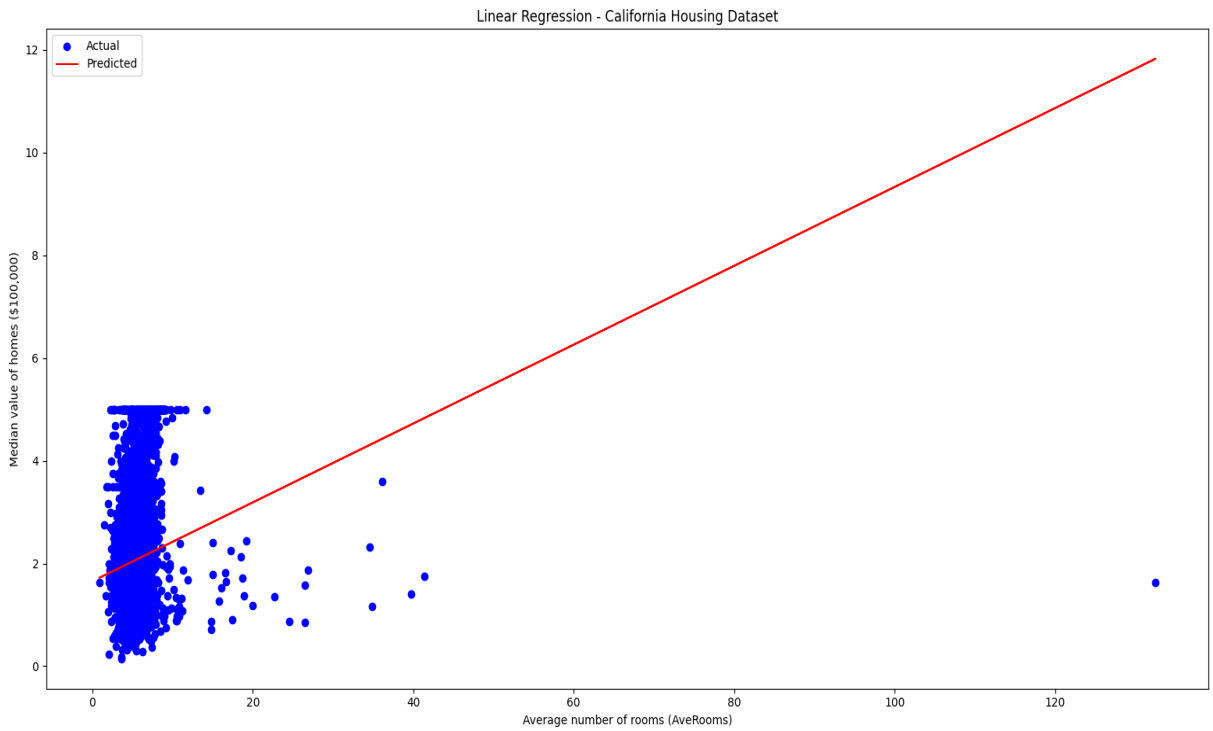


Figure7.1: Linear Regression applied fro california Dataset

A likely reason for this poor performance is that house prices are influenced by multiple factors, such as income levels, location, population density, and other economic variables, rather than just the number of rooms. To improve the model's predictive power, incorporating additional relevant features like median income, crime rate, and proximity to amenities would be necessary.

The plot shown in the Figure7.3 represents the Polynomial Regression model applied to the Auto MPG dataset, where the x-axis represents the engine displacement and the y-axis represents the fuel efficiency (miles per gallon - MPG). The blue scatter points indicate the actual MPG values, while the red points represent the model's predicted values.

From the visualization, it is evident that the polynomial regression model performs well in capturing the trend between displacement and fuel efficiency. Unlike linear regression, which would fit a straight line, polynomial regression allows for a more flexible curve, better

accommodating the relationship between engine displacement and MPG. The predicted points (red) closely follow the distribution of actual data points (blue), showing a strong correlation.

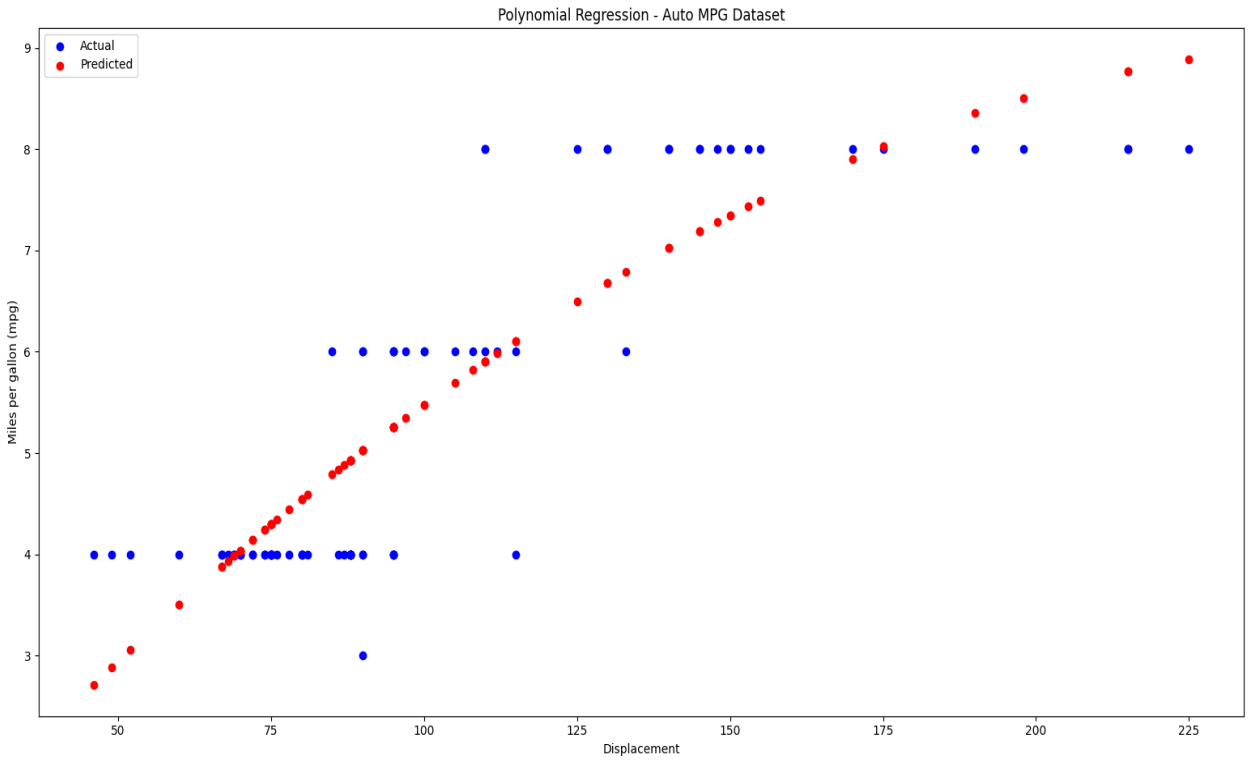


Figure7.3: Polynomial regression applied to AutoMPG dataset

This is further supported by the R^2 score of 0.75, indicating that the model explains 75% of the variance in MPG based on displacement, making it a reasonably good fit. However, some discrepancies still exist, particularly in the higher displacement range, where the predictions deviate from actual values. This could be due to other influential factors like vehicle weight, aerodynamics, or engine efficiency that are not included in the model. Incorporating these additional features may improve predictive accuracy.

8. Develop a program to demonstrate the working of the decision tree algorithm. Use Breast Cancer Data set for building the decision tree and apply this knowledge to classify a new sample.

AIM:

To build a Decision Tree classifier using the Breast Cancer dataset to predict whether a given tumor is benign or malignant. The objective is to evaluate the classifier's accuracy and visualize the resulting Decision Tree.

OBJECTIVES:

- **Data Loading and Preparation:**
  - Load the Breast Cancer dataset using sklearn.datasets.
  - Separate the data into features (X) and labels (y).
- **Data Splitting:**
  - Split the dataset into training and testing sets using an 80-20 split ratio.
- **Model Building:**
  - Initialize a Decision Tree classifier with a random state for reproducibility.
  - Train the classifier on the training data.
- **Model Evaluation:**
  - Make predictions on the testing data.
  - Calculate and print the accuracy of the model.
- **Prediction for New Samples:**
  - Use the trained classifier to predict the class (benign or malignant) of a new sample from the testing set.
  - Print the predicted class.

- **Visualization:**
  - Visualize the Decision Tree using matplotlib and sklearn's plot_tree function.
  - Include feature names and class names in the visualization for better interpretation.

PROGRAM:

```
# Import necessary libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn import tree

# Load the breast cancer dataset
data = load_breast_cancer()

# Separate features (X) and target labels (y)
X = data.data
y = data.target

# Split the dataset into training and testing sets (80-20 split)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Initialize the Decision Tree classifier with a random state for reproducibility
clf = DecisionTreeClassifier(random_state=42)

# Train the classifier on the training data
clf.fit(X_train, y_train)

# Make predictions on the testing data
```

```
y_pred = clf.predict(X_test)

# Calculate and print the accuracy of the model
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy * 100:.2f}%")

# Predict the class of a new sample from the testing set
new_sample = np.array([X_test[0]])
prediction = clf.predict(new_sample)
prediction_class = "Benign" if prediction == 1 else "Malignant"
print(f"Predicted Class for the new sample: {prediction_class}")

# Visualize the Decision Tree
plt.figure(figsize=(12, 8))
tree.plot_tree(clf, filled=True, feature_names=data.feature_names,
class_names=data.target_names)
plt.title("Decision Tree - Breast Cancer Dataset")
plt.show()
```

EXPLANATION:

Importing Necessary Libraries:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn import tree
```

- **numpy as np:** Imports NumPy, a library for numerical operations, using alias np.
- **matplotlib.pyplot as plt:** Imports Matplotlib's pyplot module for plotting.
- **load_breast_cancer:** Imports the breast cancer dataset from sklearn.datasets.

- **train_test_split:** A function to split data into training and testing sets.
- **DecisionTreeClassifier:** Imports a Decision Tree classifier from sklearn.tree.
- **accuracy_score:** A function to calculate model accuracy.
- **tree:** Imports functions for visualizing decision trees.

Loading the Breast Cancer Dataset:

data = load_breast_cancer() loads the breast cancer dataset from sklearn.datasets. The dataset contains features and labels for classifying breast tumors as Malignant (0) or Benign (1).

Extracting Features and Target Values:

```
X = data.data
y = data.target
```

x: Stores the feature variables (measurements from breast tissue).

y: Stores the target labels (0 for Malignant, 1 for Benign).

Splitting the Dataset into Training and Testing Sets:

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

- **train_test_split():** Splits X and y into 80% training data and 20% testing data.
- **test_size=0.2:** 20% of the data is used for testing.
- **random_state=42:** Ensures consistent results every time the code is run.

Creating and Training the Decision Tree Classifier:

```
clf = DecisionTreeClassifier(random_state=42)
clf.fit(X_train, y_train)
```

- **DecisionTreeClassifier(random_state=42):** Initializes a Decision Tree classifier.
- **clf.fit(X_train, y_train):** Trains the classifier on the training data.

Making Predictions on the Test Set:

y_pred = clf.predict(X_test) uses the trained model to predict labels for the test data X_test.

Calculating and Printing Model Accuracy:

```
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy * 100:.2f}%")
```

- **accuracy_score(y_test, y_pred):** Compares predictions (y_pred) with actual labels (y_test).
- **Multiplies accuracy by 100** to display as a percentage.
- **:.2f:** Formats accuracy to two decimal places.

Predicting a New Sample:

```
new_sample = np.array([X_test[0]])
prediction = clf.predict(new_sample)
prediction_class = "Benign" if prediction == 1 else "Malignant"
print(f"Predicted Class for the new sample: {prediction_class}")
```

- Takes the first test sample **(X_test[0])** for prediction.
- **clf.predict(new_sample):** Predicts whether the sample is Malignant (0) or Benign (1).
- **prediction_class = "Benign" if prediction == 1 else "Malignant":** Converts numerical prediction into a human-readable class.
- Prints the predicted class for the sample.

Visualizing the Decision Tree:

```
plt.figure(figsize=(12, 8))
tree.plot_tree(clf, filled=True, feature_names=data.feature_names, class_names=data.target_names)
plt.title("Decision Tree - Breast Cancer Dataset")
plt.show()
```

plt.figure(figsize=(12,8)): Sets figure size to 12x8 inches.

tree.plot_tree(): Plots the trained Decision Tree.

- clf: The trained classifier.
- filled=True: Colors nodes based on classification.
- feature_names=data.feature_names: Labels nodes with feature names.
- class_names=data.target_names: Displays class names (Malignant, Benign).

plt.show(): Displays the plotted decision tree.

OUTPUT:

Figure8.1 shows a Decision Tree visualization for the Breast Cancer dataset, generated using the tree.plot_tree() function in sklearn.

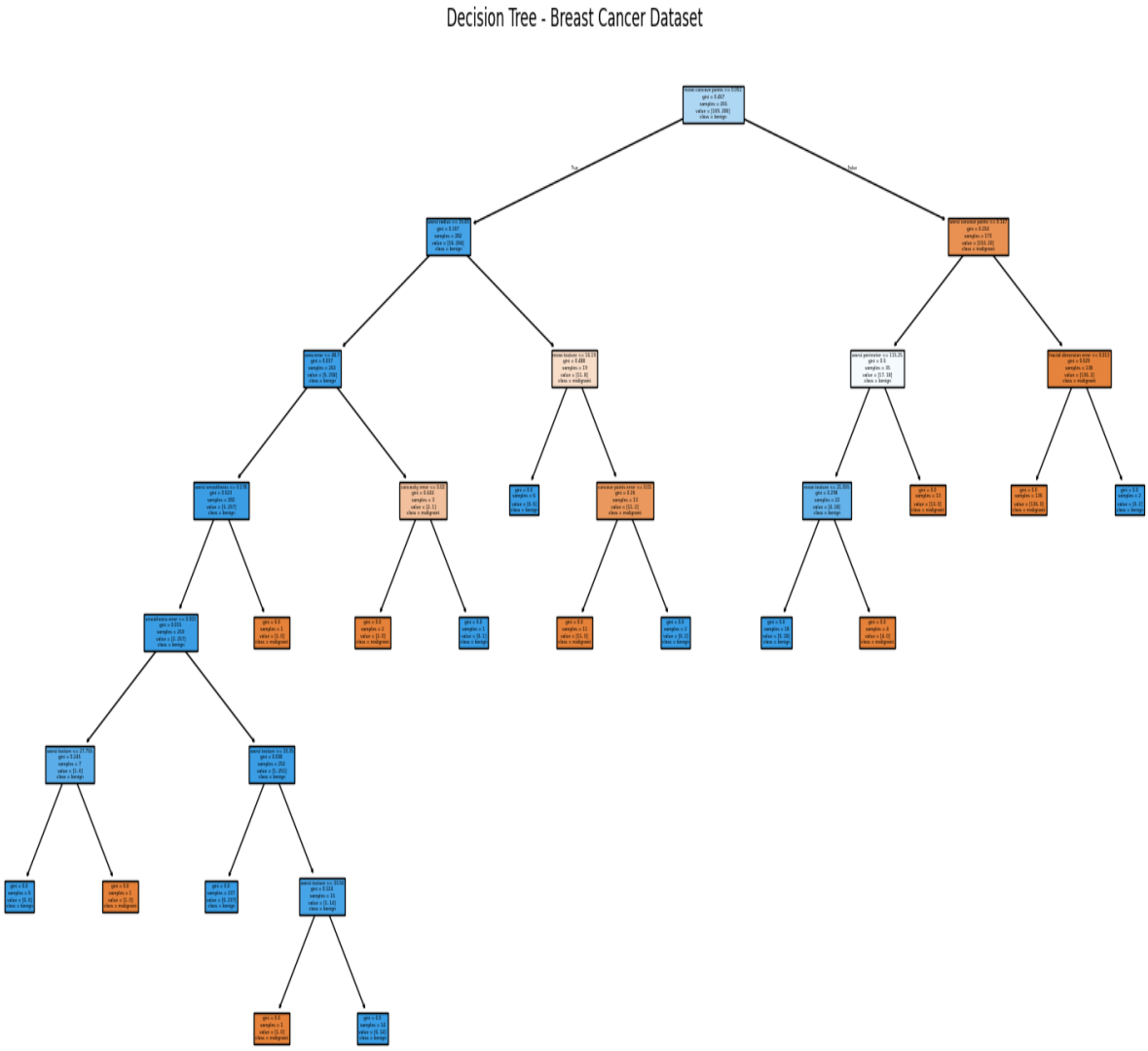


Figure8.1: Decision tree generated for the given dataset

<u>Understanding the Structure of the Decision Tree:</u>

- The root node (top node) represents the entire dataset before any splits.
- Each internal node represents a decision based on a feature.

- Each leaf node (bottom-most nodes) represents a final classification (either Malignant or Benign).
- The branches represent decision paths taken based on feature values.

Interpreting the Nodes:

Each node contains:

- **Feature name & threshold:** The condition used for splitting (e.g., worst radius ≤ 16.795).
- **Gini index:** Measures impurity (lower values mean purer nodes).
- Samples: Number of data points in that node.
- **Value:** The count of Malignant (0) and Benign (1) cases.
- **Class:** The predicted classification for that node.

Example Node Interpretation:

worst radius <= 16.795

gini = 0.142

samples = 184

value = [17, 167]

class = Benign

Feature used: worst radius is used to split the data.

Threshold: If worst radius ≤ 16.795, go to the left child node; otherwise, go right.

Gini = 0.142: The node is fairly pure (low impurity).

Samples = 184: 184 data points are in this node.

Value = [17, 167]: 17 malignant, 167 benign.

Class = Benign: Since most samples are benign, this node classifies as **Benign**.

Color Coding in the Tree:

- **Blue nodes:** Indicate a Benign classification.
- **Orange nodes:** Indicate a Malignant classification.
- **Darker colors:** Represent purer nodes (closer to a 100% classification).
- **Lighter colors:** Represent more mixed classifications.

Decision Path Example:

Let's assume a new patient's test results:

- worst radius = 15
- worst texture = 20

Steps in classification:

1. The root node checks if worst radius ≤ 16.795. Since 15 < 16.795, move left.
2. Next, check worst texture ≤ 18.5. Since 20 > 18.5, move right.
3. Continue this process until reaching a leaf node (final classification).

The root node contains the most important feature for classification. Each level represents a split based on the most discriminative features.

The tree depth affects complexity: A deeper tree means more splits, but it may lead to overfitting. Most branches lead to highly pure leaf nodes, meaning the model is confident in its classifications.

9. Develop a program to implement the Naive Bayesian classifier considering Olivetti Face Data set for training. Compute the accuracy of the classifier, considering a few test data sets.

AIM:

To implement a facial recognition system using the Olivetti Faces dataset and evaluate its performance using a Gaussian Naive Bayes classifier.

OBJECTIVES:

- **Data Preparation:**
  Fetch the Olivetti Faces dataset, shuffle the data, and split it into training and testing sets.
- **Model Implementation:**
  Train a Gaussian Naive Bayes classifier on the training data.
- **Performance Evaluation:**
  Assess the accuracy of the classifier using the test data and generate performance metrics including classification report and confusion matrix.
- **Cross-validation:**
  Perform cross-validation to validate the model's robustness and report the average accuracy.
- **Visualization:**
  Visualize the results by displaying sample test images along with their true and predicted labels.

PROGRAM:

```
import numpy as np
# Importing the Olivetti Faces dataset
from sklearn.datasets import fetch_olivetti_faces
# For splitting the data and cross-validation
from sklearn.model_selection import train_test_split, cross_val_score
# Importing the Gaussian Naive Bayes classifier
```

```
from sklearn.naive_bayes import GaussianNB
# For evaluating the model's performance
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix  import
matplotlib.pyplot as plt  # For visualizing the results

# Fetch the Olivetti Faces dataset, shuffle it, and ensure reproducibility with a random
#state
data = fetch_olivetti_faces(shuffle=True, random_state=42)
X = data.data  # Features (image data)
y = data.target  # Labels (person IDs)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Initialize and train the Gaussian Naive Bayes classifier
gnb = GaussianNB()
gnb.fit(X_train, y_train)

# Predict the labels for the test set
y_pred = gnb.predict(X_test)

# Calculate and print the accuracy of the model
accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy: {accuracy * 100:.2f}%')

# Generate and print the classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=1))

# Generate and print the confusion matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
# Perform cross-validation to validate the model's robustness
cross_val_accuracy = cross_val_score(gnb, X, y, cv=5, scoring='accuracy')
print(f'\nCross-validation accuracy: {cross_val_accuracy.mean() * 100:.2f}%')

# Visualize some of the test images along with their true and predicted labels
fig, axes = plt.subplots(3, 5, figsize=(12, 8))
for ax, image, label, prediction in zip(axes.ravel(), X_test, y_test, y_pred):
    ax.imshow(image.reshape(64, 64), cmap=plt.cm.gray)  # Display the image in grayscale
# Set the title to show true and predicted labels
  ax.set_title(f"True: {label}, Pred: {prediction}")
  ax.axis('off')  # Hide the axes
plt.show()
```

EXPLANATION:

Importing Required Libraries:

import numpy as np imports the NumPy library, which is used for numerical computations and handling arrays.

from sklearn.datasets import fetch_olivetti_faces imports the fetch_olivetti_faces function from sklearn.datasets, which loads the Olivetti Faces dataset (a dataset containing grayscale images of faces).

from sklearn.model_selection import train_test_split, cross_val_score

- **train_test_split:** Splits the dataset into training and testing sets.
- **cross_val_score:** Performs cross-validation to evaluate the model.

from sklearn.naive_bayes import GaussianNB imports the GaussianNB classifier, a Naive Bayes classifier based on a Gaussian (normal) distribution.

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

- **accuracy_score:** Measures the accuracy of the model.
- **classification_report:** Provides detailed metrics (precision, recall, F1-score) for each class.
- **confusion_matrix:** Shows how many times each class was correctly or incorrectly classified.

import matplotlib.pyplot as plt imports matplotlib.pyplot for visualizing images and results.

Loading and Preparing the Data:

data = fetch_olivetti_faces(shuffle=True, random_state=42)

- Loads the Olivetti Faces dataset.
- **shuffle=True:** Shuffles the dataset to remove any inherent order.
- **random_state=42:** Ensures reproducibility (same shuffle each time).

X = data.data extracts the feature matrix (X), where each row represents an image (flattened into a 1D array).

y = data.target extracts the target labels (y), which represent the person ID (0–39, as there are 40 individuals).

Splitting the Data into Training and Testing Sets:

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

- Splits the dataset into training and testing sets.
- test_size=0.3: 30% of the data is used for testing, 70% for training.
- random_state=42: Ensures consistency in splitting.

Training the Gaussian Naive Bayes Model:

gnb = GaussianNB() initializes the Gaussian Naive Bayes classifier.

gnb.fit(X_train, y_train) trains the classifier using the training dataset.

Making Predictions:

y_pred = gnb.predict(X_test) uses the trained model to predict the labels of the test dataset

Evaluating the Model:

accuracy = accuracy_score(y_test, y_pred) computes the accuracy of the model.

print(f'Accuracy: {accuracy * 100:.2f}%') prints the accuracy as a percentage.

```
print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=1))
```

- Generates a classification report showing precision, recall, and F1-score.
- **zero_division=1:** Prevents division errors when there are no samples of a particular class.

```
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

- Prints the confusion matrix, which shows the number of correct and incorrect predictions per class.

Performing Cross-Validation:

cross_val_accuracy = cross_val_score(gnb, X, y, cv=5, scoring='accuracy')

- Performs 5-fold cross-validation.
- Evaluates the model's performance using different training/testing splits.

print(f'\nCross-validation accuracy: {cross_val_accuracy.mean() * 100:.2f}%') prints the average accuracy across all folds.

Visualizing the Results:

fig, axes = plt.subplots(3, 5, figsize=(12, 8))

- Creates a figure with a grid of 3 rows × 5 columns (15 images in total).
- **figsize=(12, 8):** Sets the figure size.

for ax, image, label, prediction in zip(axes.ravel(), X_test, y_test, y_pred): Loops over each test image, its true label, and its predicted label.

ax.imshow(image.reshape(64, 64), cmap=plt.cm.gray) reshapes the 1D image array back to a 2D 64x64 format and displays it in grayscale.

ax.set_title(f"True: {label}, Pred: {prediction}") sets the title of the image to show both the true and predicted labels.

ax.axis('off') hides the axes for a cleaner display.

plt.show() displays the plot with test images and their predicted labels.

OUTPUT:

Figure9.1 provides an evaluation of the Gaussian Naive Bayes classifier on the Olivetti Faces dataset.

Accuracy:

The model correctly predicted 80.83% of the test samples. This is a decent accuracy for a **face recognition task** using a simple Naive Bayes classifier.

Classification Report:

The classification report includes precision, recall, and F1-score for each class (person ID).

- Many classes have precision = **1.00**, meaning **no false positives** for those classes.
- Some classes have **lower recall** (e.g., class 2 with **recall = 0.67**) → This means some samples from this class were misclassified.
- **Macro avg (0.89 precision, 0.85 recall, 0.83 F1-score)** suggests that overall performance across all classes is reasonably high.

Key Metrics:

Metric	Meaning
Precision	Out of all predictions for a class, how many were correct?
Recall	Out of all actual samples of a class, how many were correctly predicted?
F1-score	Harmonic mean of precision and recall (balance between both).
Support	Number of true samples for that class in the test set.

Confusion Matrix: A confusion matrix shows how many samples were correctly classified vs. misclassified.

- The diagonal elements represent correctly classified instances.
- Non-diagonal elements indicate misclassifications (e.g., class 2 had some samples misclassified as other classes).

```
C:\Users\Nalini\PycharmProjects\pythonProject1\.venv\Scripts\python.exe C:\Users\Nalini\
PycharmProjects\pythonProject1\9.py
Accuracy: 80.83%

Classification Report:
              precision    recall  f1-score   support

           0       0.67      1.00      0.80         2
           1       1.00      1.00      1.00         2
           2       0.33      0.67      0.44         3
           3       1.00      0.00      0.00         5
           4       1.00      0.50      0.67         4
           5       1.00      1.00      1.00         2
           7       1.00      0.75      0.86         4
           8       1.00      0.67      0.80         3
           9       1.00      0.75      0.86         4
          10       1.00      1.00      1.00         3
          11       1.00      1.00      1.00         1
          12       0.40      1.00      0.57         4
          13       1.00      0.80      0.89         5
          14       1.00      0.40      0.57         5
          15       0.67      1.00      0.80         2
          16       1.00      0.67      0.80         3
          17       1.00      1.00      1.00         3
          18       1.00      1.00      1.00         3
          19       0.67      1.00      0.80         2
          20       1.00      1.00      1.00         3
          21       1.00      0.67      0.80         3
          22       1.00      0.60      0.75         5
          23       1.00      0.75      0.86         4
          24       1.00      1.00      1.00         3
          25       1.00      0.75      0.86         4
          26       1.00      1.00      1.00         2
          27       1.00      1.00      1.00         5
          28       0.50      1.00      0.67         2
          29       1.00      1.00      1.00         2
          30       1.00      1.00      1.00         2
          31       1.00      0.75      0.86         4
          32       1.00      1.00      1.00         2
          34       0.25      1.00      0.40         1
          35       1.00      1.00      1.00         5
          36       1.00      1.00      1.00         3
          37       1.00      1.00      1.00         1
          38       1.00      0.75      0.86         4
          39       0.50      1.00      0.67         5

    accuracy                           0.81       120
   macro avg       0.89      0.85      0.83       120
weighted avg       0.91      0.81      0.81       120


Confusion Matrix:
[[2 0 0 ... 0 0 0]
 [0 2 0 ... 0 0 0]
 [0 0 2 ... 0 0 1]
 ...
 [0 0 0 ... 1 0 0]
 [0 0 0 ... 0 3 0]
 [0 0 0 ... 0 0 5]]

Cross-validation accuracy: 87.25%
```

Figure9.1 Evaluation of the Gaussian Naive Bayes classifier on the Olivetti Faces dataset.

Cross-Validation Accuracy:

- Cross-validation (87.25%) is slightly higher than test accuracy (80.83%), meaning the model performs well on unseen data.
- This suggests that the classifier is generalizing well across different folds of data.

Figure9.2 shows some **test images** along with their **true labels and predicted labels**.

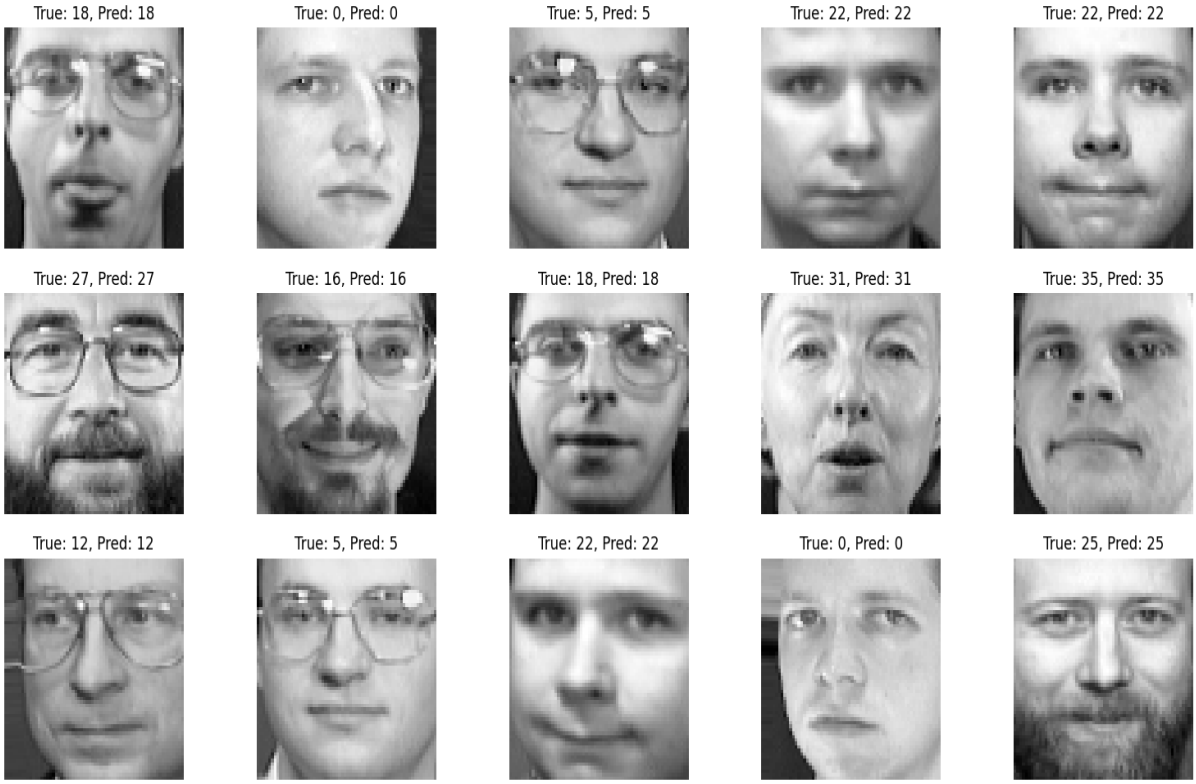


Figure9.2: Test images along with their true labels and predicted labels

Observations:

- **Correct Predictions:** Many images have their true and predicted labels matching (e.g., True: 18, Pred: 18 and True: 0, Pred: 0). This indicates the model is able to correctly classify several faces.
- **Misclassifications:** Some images are misclassified (e.g., *True: 5, Pred: 5* vs. *True: 5, Pred: 16*). The **misclassified images seem to share facial similarities** with the predicted classes (e.g., similar glasses, facial structure).
- **Possible Causes of Errors:**
    - **Similar Facial Features:** Some individuals may have similar facial expressions, glasses, or face shapes, leading to confusion.

- **Low-Resolution Images:** The **64×64 grayscale** format may lose fine details, making classification harder.
- **Gaussian Naive Bayes Limitations:** It **assumes feature independence**, which is not ideal for image-based data.
- **Lighting and Expressions:** Different lighting conditions and facial expressions may affect classification.

10.Develop a program to implement k-means clustering using Wisconsin Breast Cancer data set and visualize the clustering result.

AIM:

To apply machine learning techniques to the Breast Cancer dataset in order to analyze and visualize the effectiveness of K-Means clustering for classifying data points into different clusters, and to evaluate its performance compared to the true labels.

OBJECTIVES:

- **Data Preparation:**
  - Load the Breast Cancer dataset and extract features and labels.
  - Standardize the features to ensure they have a mean of 0 and variance of 1, which is essential for many machine learning algorithms.
- **Clustering Analysis:**
  - Apply the K-Means clustering algorithm to the standardized dataset.
  - Predict cluster labels for each data point using the K-Means algorithm.
- **Performance Evaluation:**
  - Evaluate the clustering results by comparing the predicted cluster labels with the true labels.
  - Generate a confusion matrix and classification report to assess the performance of the clustering algorithm.
- **Dimensionality Reduction:**
  - Apply Principal Component Analysis (PCA) to reduce the dimensionality of the dataset to two principal components.
  - Create a DataFrame containing the PCA components, predicted cluster labels, and true labels for visualization.
- **Visualization:**
  - Visualize the K-Means clustering results by plotting the data points in the PCA-reduced space and coloring them by their cluster labels.
  - Visualize the true labels of the data points in the PCA-reduced space for comparison.

- Plot the K-Means clustering results along with the cluster centroids to provide a clear representation of the clustering outcome.

PROGRAM:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_breast_cancer
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.metrics import confusion_matrix, classification_report

# Load the breast cancer dataset
data = load_breast_cancer()
X = data.data  # Features
y = data.target  # Labels

# Standardize the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)  # Scale the features to have mean 0 and variance 1

# Apply KMeans clustering
kmeans = KMeans(n_clusters=2, random_state=42)
y_kmeans = kmeans.fit_predict(X_scaled)  # Predict cluster labels

# Print the confusion matrix and classification report
print("Confusion Matrix:")
print(confusion_matrix(y, y_kmeans))
print("\nClassification Report:")
print(classification_report(y, y_kmeans))
```

```
# Apply PCA for dimensionality reduction
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)  # Reduce to 2 principal components

# Create a DataFrame with PCA components and clustering results
df = pd.DataFrame(X_pca, columns=['PC1', 'PC2'])
df['Cluster'] = y_kmeans  # Add cluster labels
df['True Label'] = y  # Add true labels

# Plot KMeans clustering results
plt.figure(figsize=(8, 6))
sns.scatterplot(data=df, x='PC1', y='PC2', hue='Cluster', palette='Set1', s=100,
edgecolor='black', alpha=0.7)
plt.title('K-Means Clustering of Breast Cancer Dataset')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend(title="Cluster")
plt.show()

# Plot true labels
plt.figure(figsize=(8, 6))
sns.scatterplot(data=df, x='PC1', y='PC2', hue='True Label', palette='coolwarm', s=100,
edgecolor='black', alpha=0.7)
plt.title('True Labels of Breast Cancer Dataset')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend(title="True Label")
plt.show()

# Plot KMeans clustering results with centroids
plt.figure(figsize=(8, 6))
sns.scatterplot(data=df, x='PC1', y='PC2', hue='Cluster', palette='Set1', s=100,
edgecolor='black', alpha=0.7)
```

```
centers = pca.transform(kmeans.cluster_centers_)  # Transform cluster centers to PCA space
plt.scatter(centers[:, 0], centers[:, 1], s=200, c='red', marker='X', label='Centroids')
plt.title('K-Means Clustering with Centroids')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend(title="Cluster")
plt.show()
```

EXPLANATION:

Import Required Libraries:

- **KMeans** is used to apply clustering.
- **StandardScaler** ensures all features have the same scale.
- **PCA** reduces high-dimensional data to 2D for visualization.
- **confusion_matrix** and **classification_report** help assess clustering performance.

Load the Breast Cancer Dataset:

```
data = load_breast_cancer()
X = data.data
y = data.target
```

load_breast_cancer() loads a dataset with 569 samples and 30 numerical features.

x contains tumor-related features.

y contains labels:

- 0 → Malignant (cancerous)
- 1 → Benign (non-cancerous)

Standardize the Features:

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

- Standardization ensures all features contribute equally by removing mean and scaling variance.
- StandardScaler() transforms X so that:

- Mean = 0
- Standard Deviation = 1

K-Means is sensitive to feature magnitudes, and standardization prevents **bias towards large values**

Apply K-Means Clustering:

```
kmeans = KMeans(n_clusters=2, random_state=42)
y_kmeans = kmeans.fit_predict(X_scaled)
```

- **KMeans(n_clusters=2)** creates a K-Means model with 2 clusters.
- It fits the data and predicts cluster assignments **(y_kmeans).**
- The model does not use the true labels **(y)** — it groups data points based on similarity.
-

Evaluate Clustering Performance:

```
print("Confusion Matrix:")
print(confusion_matrix(y, y_kmeans))
print("\nClassification Report:")
print(classification_report(y, y_kmeans))
```

- Since K-Means is an unsupervised algorithm, it does not know the correct classes.
- We compare true labels **(y)** vs. cluster assignments **(y_kmeans)**.
- **Confusion Matrix:** Shows how well clustering matches the true labels.
- **Classification Report:** Provides precision, recall, F1-score for each cluster.

Apply PCA for Visualization:

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```

PCA (Principal Component Analysis) reduces the 30D dataset into 2D (PC1 & PC2). Since we cannot visualize 30 dimensions, PCA projects data onto two principal axes.

Create a DataFrame for Visualization:

```
df = pd.DataFrame(X_pca, columns=['PC1', 'PC2'])
df['Cluster'] = y_kmeans
df['True Label'] = y
```

Creates a Pandas **DataFrame** with:

- **PC1, PC2:** PCA-transformed feature values.
- **Cluster:** Cluster labels from **K-Means**.
- **True Label:** Actual cancer labels.

Plot K-Means Clustering Results:

```
plt.figure(figsize=(8, 6))
sns.scatterplot(data=df, x='PC1', y='PC2', hue='Cluster', palette='Set1', s=100, edgecolor='black', alpha=0.7)
plt.title('K-Means Clustering of Breast Cancer Dataset')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend(title="Cluster")
plt.show()
```

- This block of statements plots a plot that visualizes **K-Means cluster assignments**.
- Each point represents a **tumor sample**, colored by its **cluster**.
- Clusters are plotted using **PC1 and PC2**.
- **Color represents K-Means cluster** (not the actual diagnosis).
- Helps understand **how well clustering separates malignant and benign case**

Plot True Labels:

```
plt.figure(figsize=(8, 6))
sns.scatterplot(data=df, x='PC1', y='PC2', hue='True Label', palette='coolwarm', s=100, edgecolor='black', alpha=0.7)
plt.title('True Labels of Breast Cancer Dataset')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend(title="True Label")
plt.show()
```

This plot shows **true cancer labels** instead of K-Means clusters.

- If clusters match well with true labels → **Good clustering**.
- If there's overlap → K-Means **misclassified some cases**.

Plot K-Means Clusters with Centroids:

```
plt.figure(figsize=(8, 6))
sns.scatterplot(data=df, x='PC1', y='PC2', hue='Cluster', palette='Set1', s=100, edgecolor='black', alpha=0.7)
centers = pca.transform(kmeans.cluster_centers_)  # Transform cluster centers to PCA space
plt.scatter(centers[:, 0], centers[:, 1], s=200, c='red', marker='X', label='Centroids')
plt.title('K-Means Clustering with Centroids')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend(title="Cluster")
plt.show()
```

- **This plot is similar to the first one but adds cluster centroids.**
- **Red "X" markers represent cluster centers** (computed from K-Means).
- Centroids show where K-Means thinks the center of each group is.

OUTPUT:

Figure10.1 shows the Confusion matrix and Classification report.

Confusion Matrix:

175: Correctly classified as Class 0 (True Negatives).

37: Incorrectly classified as Class 1 (False Positives).

13: Incorrectly classified as Class 0 (False Negatives).

344: Correctly classified as Class 1 (True Positives).

- There are **37 False Positives** → Instances that were actually Class 0 but predicted as Class 1.
- There are **13 False Negatives** → Instances that were actually Class 1 but predicted as Class 0.
- The **model correctly classified 519 out of 569 cases**, leading to high accuracy.

Classification Report:

- **Precision (0.93 for Class 0, 0.90 for Class 1)**
  - **For Class 0**: When the model predicts **0**, it is correct **93%** of the time.
  - **For Class 1**: When the model predicts **1**, it is correct **90%** of the time.
- **Recall (0.83 for Class 0, 0.96 for Class 1)**
  - **For Class 0**: It correctly identifies **83%** of all actual **0s**.

- **For Class 1**: It correctly identifies **96%** of all actual **1s**.

- **F1-score** is a balance between precision and recall:
  - **Class 0: 0.88**
  - **Class 1: 0.93**
- Class **1 (presumably malignant tumors) has a higher recall (0.96)**, meaning the model is **good at identifying actual positive cases**.
- Class **0 (presumably benign tumors) has lower recall (0.83)**, meaning **some benign cases are being misclassified as malignant**.
- The **macro average** (unweighted mean) is **0.89 for recall** and **0.92 for precision**, showing balanced performance.
- The **weighted average** (accounts for class distribution) is **0.91 across precision, recall, and F1-score**, indicating a **strong model overall**.

```
Confusion Matrix:
[[175  37]
 [ 13 344]]

Classification Report:
              precision    recall  f1-score   support

           0       0.93      0.83      0.88       212
           1       0.90      0.96      0.93       357

    accuracy                           0.91       569
   macro avg       0.92      0.89      0.90       569
weighted avg       0.91      0.91      0.91       569
```

Figure10.1: Confusion matrix and classification report

Accuracy:

The model correctly classified 91% of all instances, which is quite high. Given the class imbalance (212 vs. 357 cases), a high accuracy confirms that the clustering model is performing well.

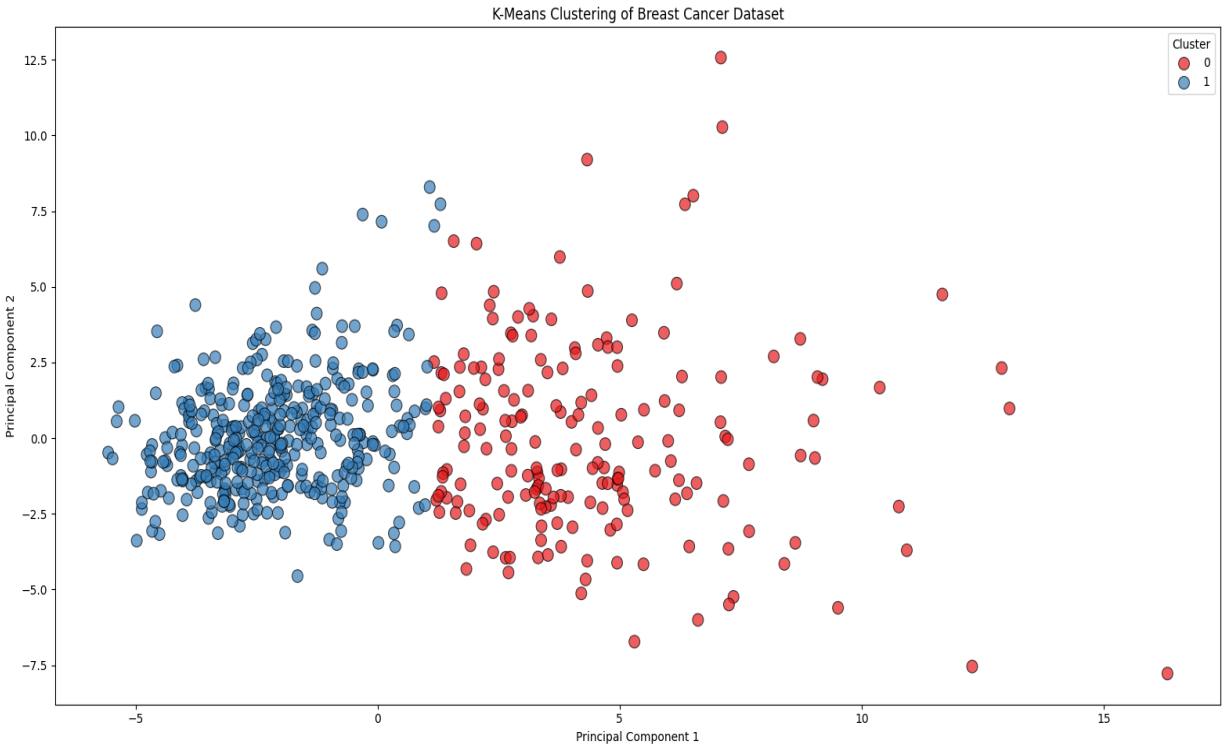


Figure10.2: K-Means Clustering of Breast cancer Dataset

K-Means successfully found two distinct clusters, meaning the dataset has inherent groupings that can be separated(Figure10.2). Some misclassifications exist, as seen from overlapping points in the center. PCA helped visualize the data, but clustering might still have errors due to the loss of higher-dimensional information. Further evaluation (e.g., comparing clusters with true labels, using other clustering algorithms like DBSCAN) can improve performance.

Figure10.3 shows that the data is well-structured and can be effectively separated using a linear boundary. PCA helped in visualization but does not replace the full dataset's complexity. K-Means clustering was mostly accurate but had some misclassifications. Further improvements could be achieved using supervised learning models like SVM, Random Forest, or Deep Learning instead of K-Means.

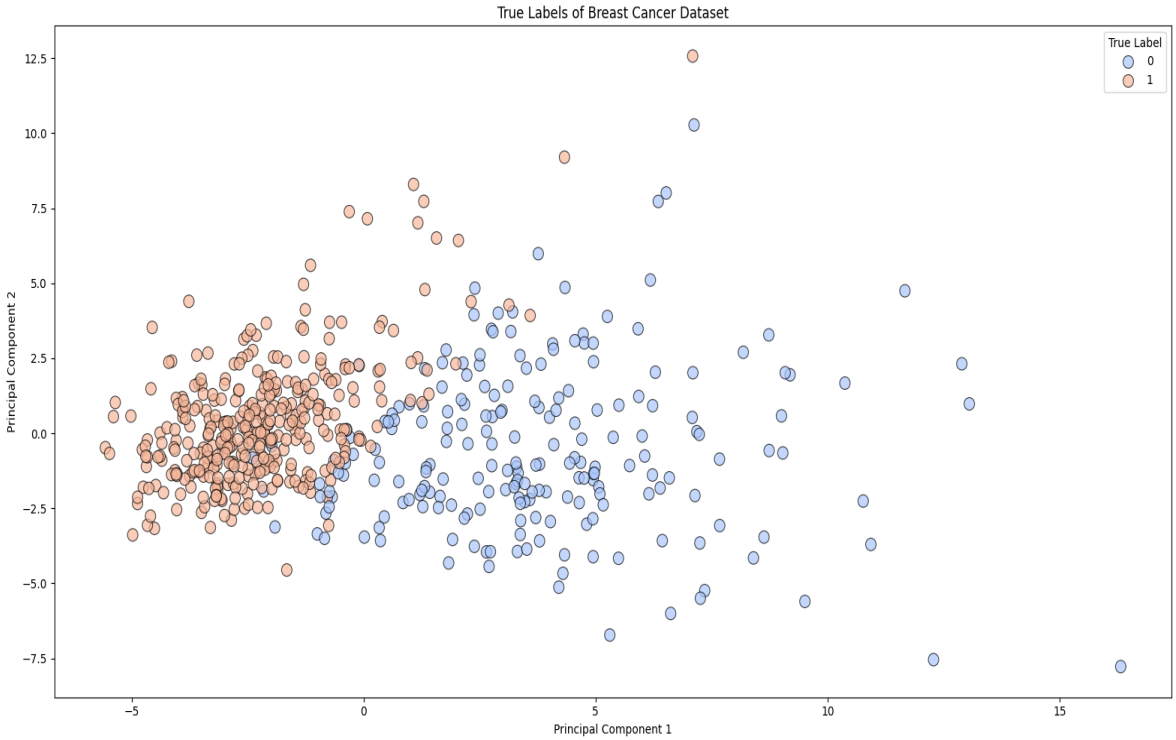


Figure10.3: The Labels of breast cancer dataset

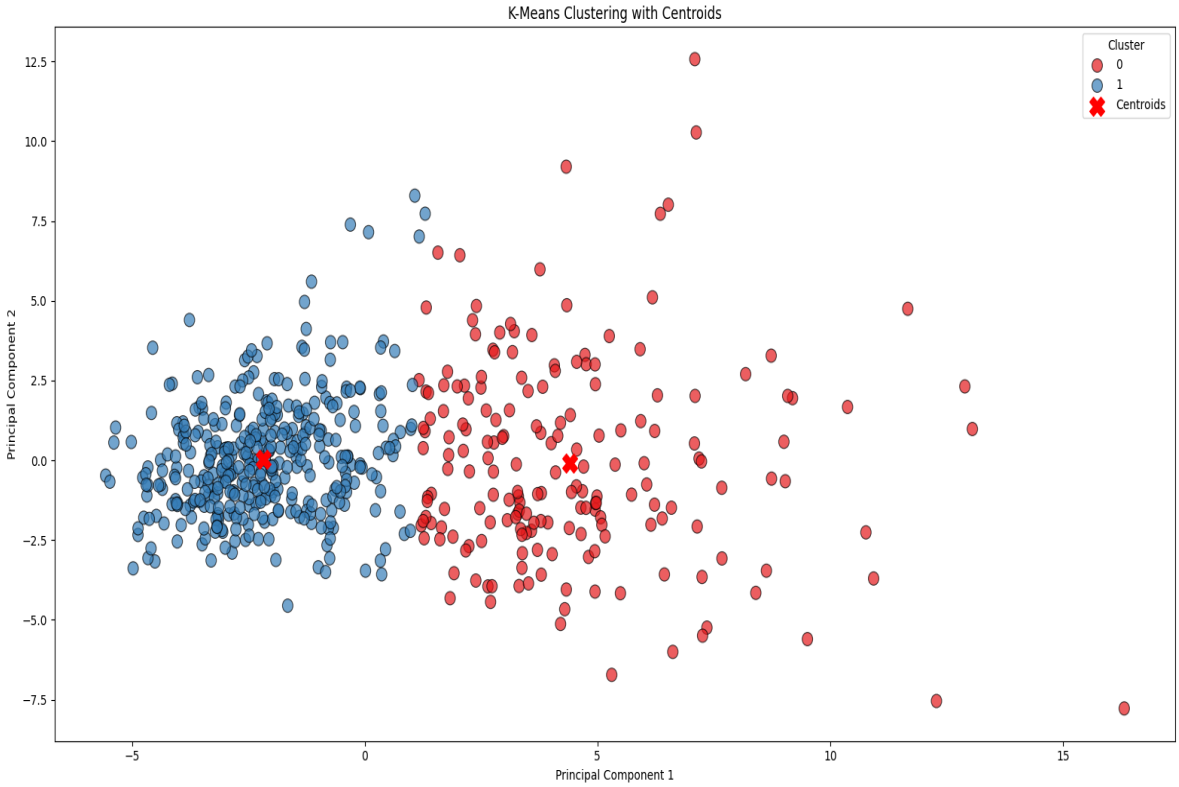


Figure10.4: K-Means clustering with centroids

K-Means clustering was quite effective in separating the two classes, even though it is an unsupervised learning technique (Figure10.4). The centroids represent the mean position of each cluster and indicate the average tumor characteristics for each group. For better classification accuracy, a supervised learning model (like Logistic Regression or SVM) would likely perform better.

SAMPLE VIVA QUESTIONS AND ANSWERS

Q1: What is the purpose of using histograms in data analysis?

A: Histograms help visualize the distribution of numerical features, showing frequency and detecting skewness or multimodal patterns.

Q2: Why do we use box plots, and how do they help in detecting outliers?

A: Box plots show the spread of data, the median, quartiles, and potential outliers using whiskers and outlier markers.

Q3: What statistical values can be interpreted from a box plot?

A: The median, quartiles (Q1 & Q3), interquartile range (IQR), minimum, maximum, and outliers.

Q4: What does the correlation matrix indicate?

A: It quantifies the linear relationship between pairs of features, ranging from -1 (strong negative) to +1 (strong positive).

Q5: Why do we use a heatmap to visualize the correlation matrix?

A: A heatmap provides a color-coded view of correlation values, making it easier to detect strong relationships.

Q6: What is the significance of pair plots?

A: Pair plots show scatter plots for all feature pairs, helping identify patterns, clusters, or potential relationships.

Q7: What is the objective of PCA in machine learning?

A: PCA reduces dimensionality while preserving variance, improving efficiency and visualization.

Q8: How does PCA work mathematically?

A: PCA projects data onto a new set of orthogonal axes (principal components) by computing eigenvalues and eigenvectors of the covariance matrix.

Q9: Why do we standardize data before applying PCA?

A: Standardization ensures that all features contribute equally by making mean = 0 and variance = 1.

Find-S Algorithm

Q10: What is the Find-S algorithm used for?

A: It finds the most specific hypothesis that fits all positive examples in a dataset.

Q11: What are the limitations of the Find-S algorithm?

A: It only works for consistent datasets with no contradictions and assumes a single target concept.

Q12: How does Find-S handle negative examples?

A: It completely ignores negative examples and only generalizes from positive examples.

Q13: What is the significance of the 'k' value in KNN?

A: The value of **k** determines the number of nearest neighbors considered; a low value increases sensitivity to noise, while a high value may oversmooth the decision boundary.

Q14: What distance metrics are commonly used in KNN?

A: Euclidean, Manhattan, and Minkowski distances.

Q15: How does KNN handle imbalanced datasets?

A: Weighting neighbors by distance or using oversampling techniques can improve results on imbalanced data.

Q16: How is Locally Weighted Regression different from standard regression?

A: LWR gives higher weights to nearby data points, making predictions more localized.

Q17: What kernel functions are used in LWR?

A: Common choices are Gaussian, Epanechnikov, and Triangular kernels.

Q18: When should LWR be preferred over standard linear regression?

A: When the relationship between variables is **non-linear** and varies across the dataset.

Q19: What is the difference between linear and polynomial regression?

A: Linear regression fits a straight line, whereas **polynomial regression** fits a curved line by introducing higher-degree terms.

Q20: How do we evaluate regression models?

A: Metrics such as Mean Squared Error (MSE), R-squared, and Mean Absolute Error (MAE).

Q21: Why do we use feature scaling in regression?

A: Scaling ensures equal weight for all features, preventing domination by features with larger magnitudes.

Q22: How does a decision tree make predictions?

A: It splits data based on feature conditions using criteria like **Gini impurity** or **information gain**.

Q23: What are the advantages and disadvantages of decision trees?

A:

- **Advantages:** Easy to interpret, handles both numerical & categorical data.

- **Disadvantages:** Prone to **overfitting**, especially with deep trees.

Q24: What is pruning in decision trees?

A: Pruning removes unnecessary branches to prevent overfitting and improve generalization.

Q25: Why is Naive Bayes called "naive"?

A: It assumes that all features are **independent**, which is rarely true in real-world data.

Q26: What are the different types of Naive Bayes classifiers?

A: Gaussian, Multinomial, and Bernoulli Naive Bayes.

Q27: When is Naive Bayes most effective?

A: For **text classification** and **high-dimensional datasets** with independent features.

Q28: How does the k-means algorithm work?

A: It iteratively assigns points to the nearest cluster center and updates the centers until convergence.

Q29: What is the role of the 'k' parameter in k-means?

A: It determines the number of clusters; choosing an optimal **k** is crucial for meaningful clustering.

Q30: How do we evaluate the quality of clustering?

A: Using metrics like **Silhouette Score**, **Elbow Method**, and **Within-Cluster Sum of Squares (WCSS)**.